



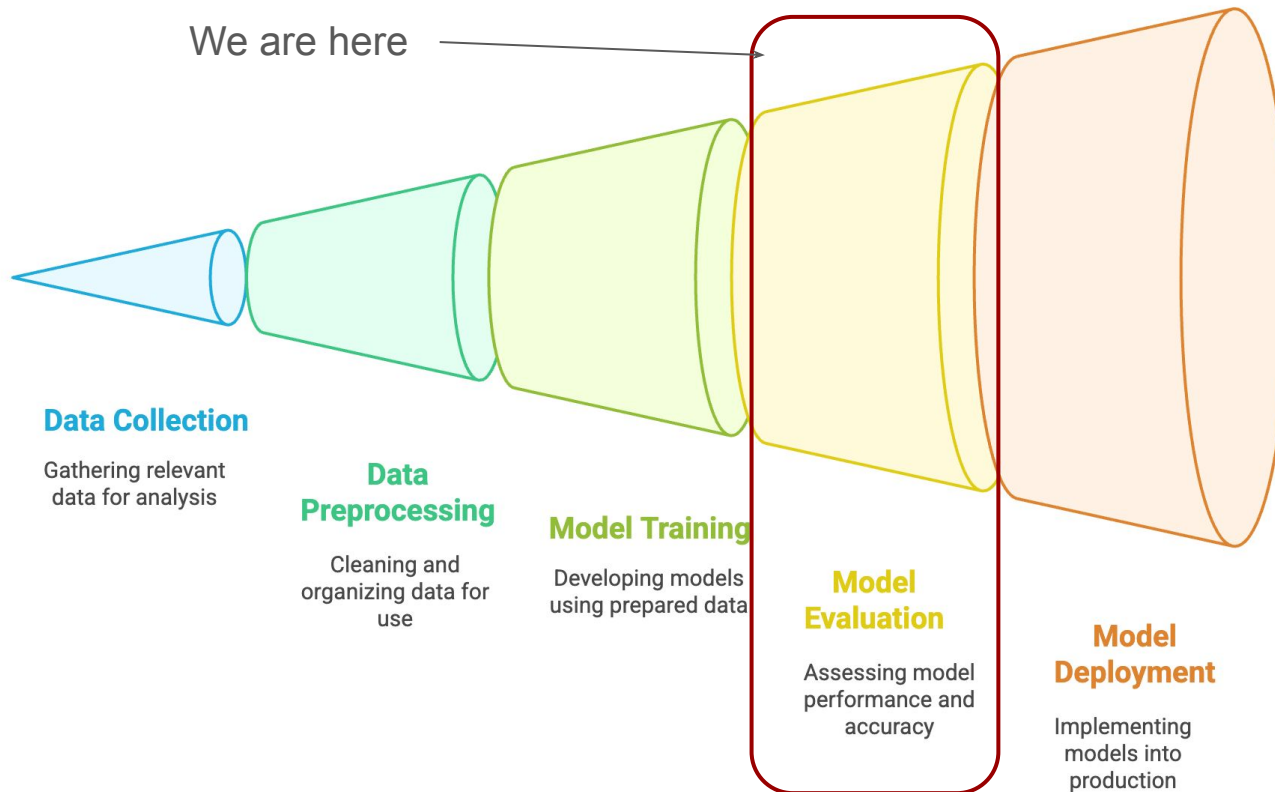
AMERICAN
UNIVERSITY
OF BEIRUT

Fall 2025

EECE 490: Introduction to Machine Learning

C6 - Statistical Model Tuning and Testing

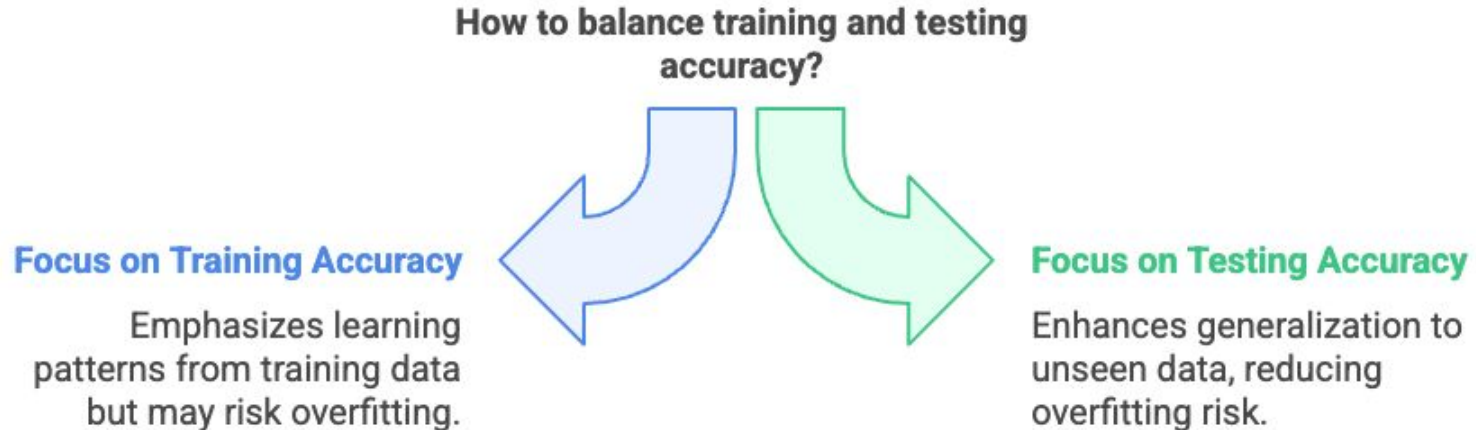
Where are we in the life cycle now?



Why do we need to test our model?

Training Accuracy Vs Testing Accuracy

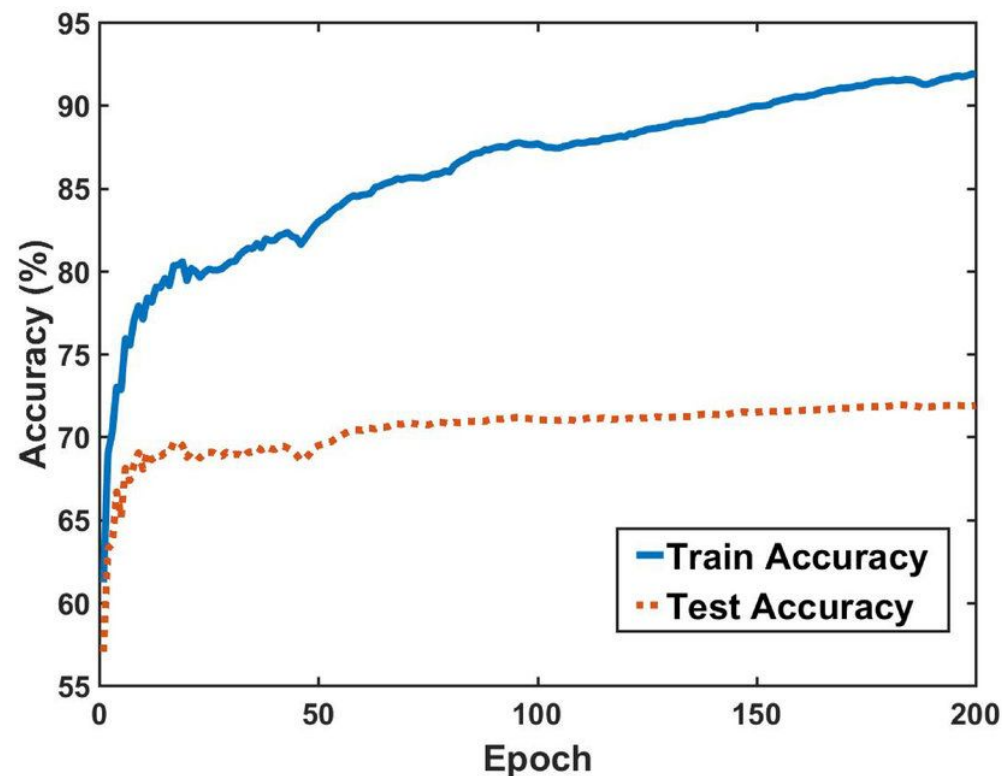
Statistical model tuning and testing are critical components in the development of predictive models. They ensure that the models not **only fit the training data** well but also **generalize effectively to unseen data**.



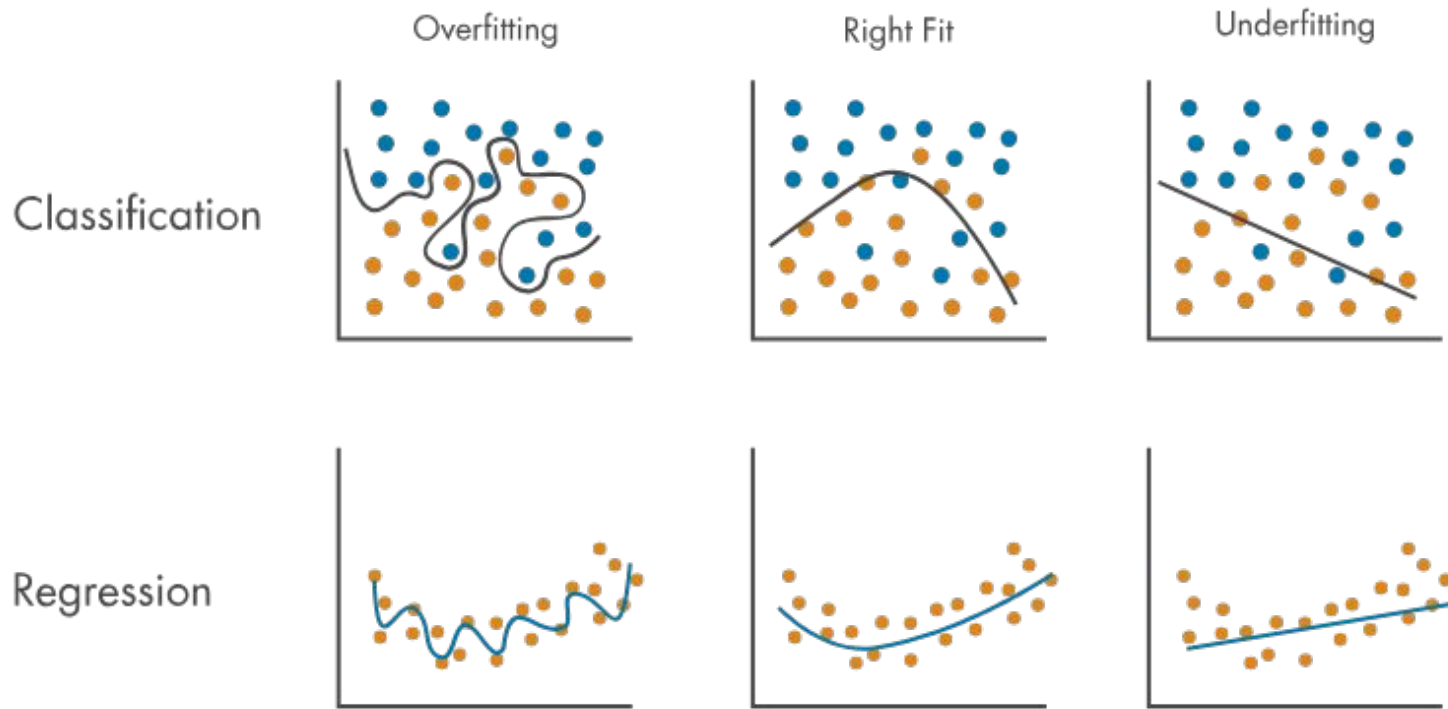
Importance of Testing

Testing allows us to assess how well the model performs on unseen data, providing a clear indication of its predictive power.

In most cases, training and testing accuracies are not similar.

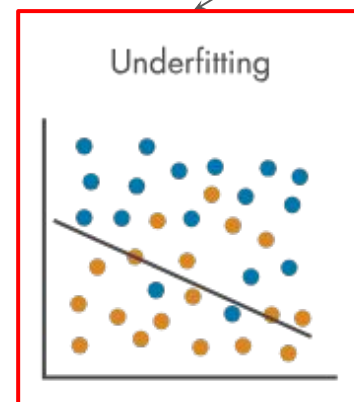
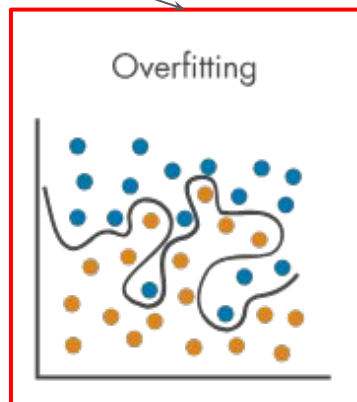


Finding the right fit



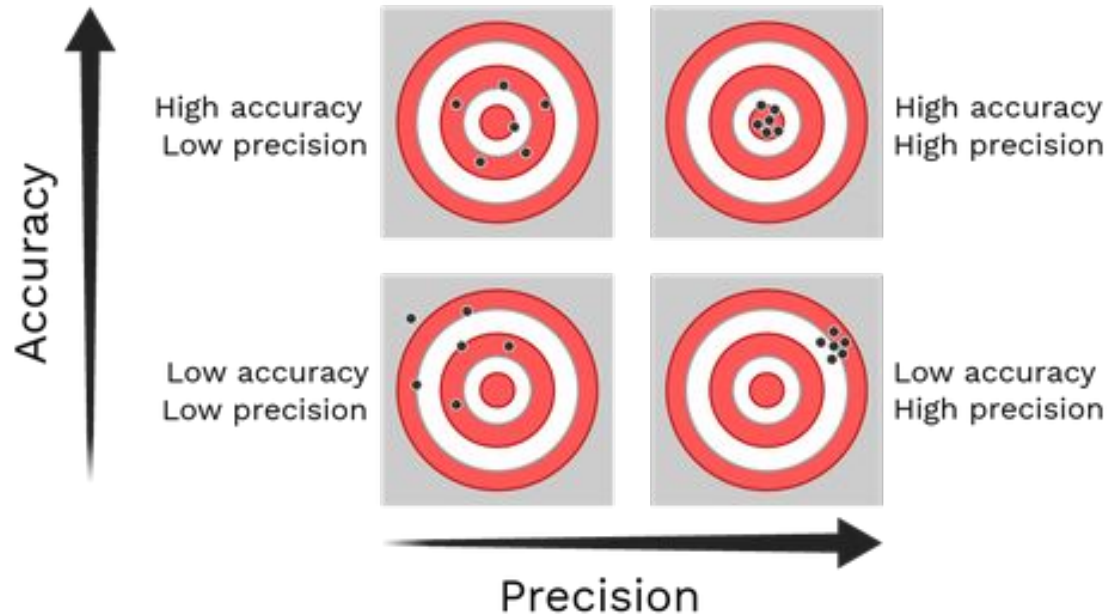
Finding the right fit

- Model is too complex
- Fits on the training data perfectly
- Does not generalize well
- Model is not too complex, and not too simple
- Models the training data with some error
- Generalizes well on unseen data
- Model is too simple
- Does not model the training data
- Does not generalize well



Finding the right fit

Ideally, we want a model with high precision and high accuracy



Bias $\leftrightarrow$ Variance Tradeoff

When a machine learning model learns from data, it tries to find patterns that help it make good predictions.

But it can make two kinds of mistakes:

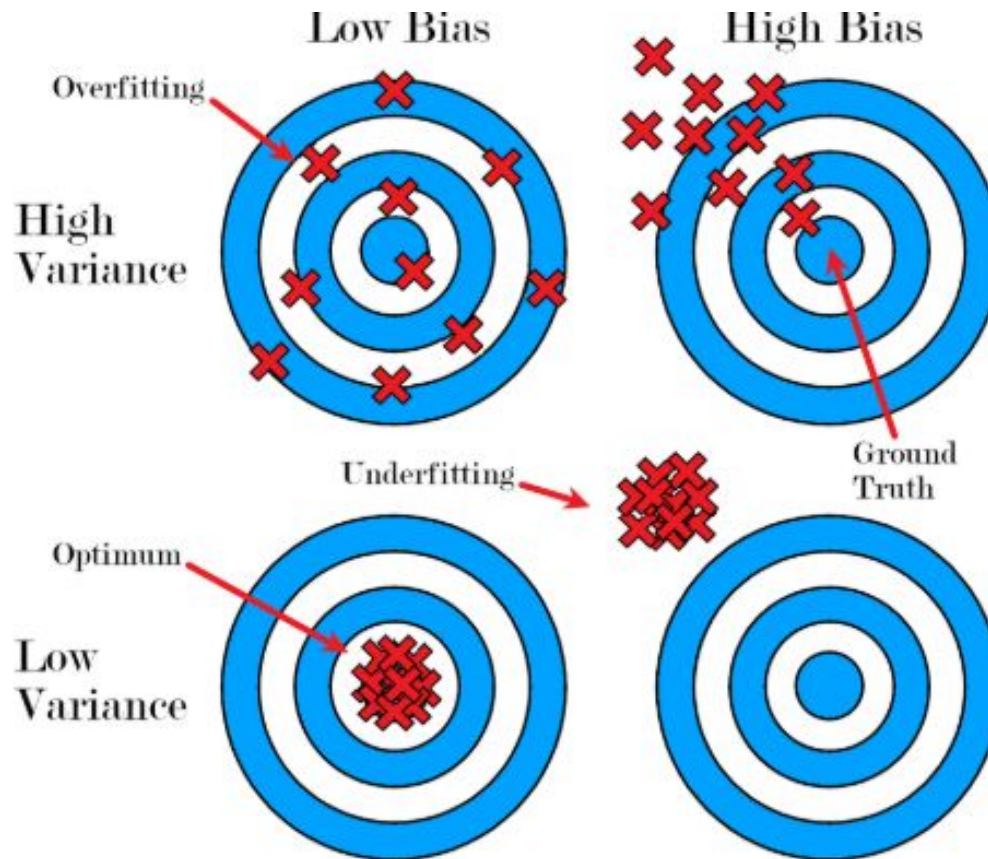
1. Bias $\rightarrow$ when the model is too simple and can't learn the real pattern.
 - It makes the same kind of mistake every time.
 - Example: using a straight line to fit a curve.
2. Variance $\rightarrow$ when the model is too sensitive to the training data.
 - It learns even the noise or random details.
 - It performs well on training data but badly on new data.

The tradeoff is this:

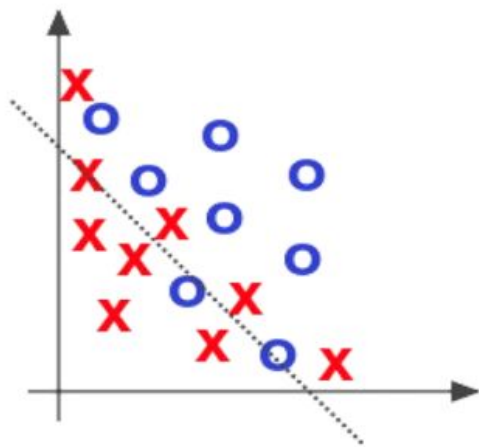
- If you make your model simpler $\rightarrow$ bias goes up, variance goes down.
- If you make your model more complex $\rightarrow$ bias goes down, variance goes up.

So you must find the sweet spot where total error ($\text{bias}^2 + \text{variance}$) is lowest.

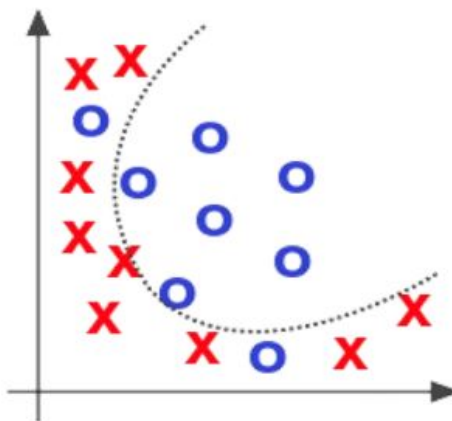
Finding the right fit



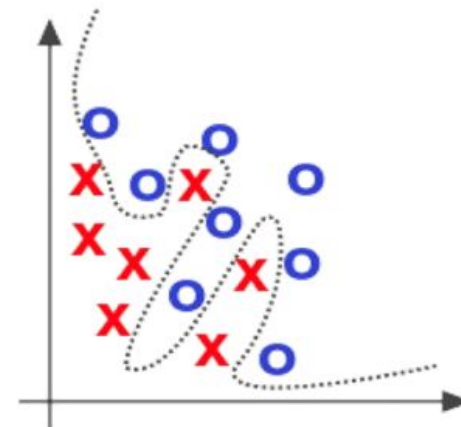
Finding the right fit



Under Fit
High bias, Low Variance



Appropriate
Bias - Variance tradeoff



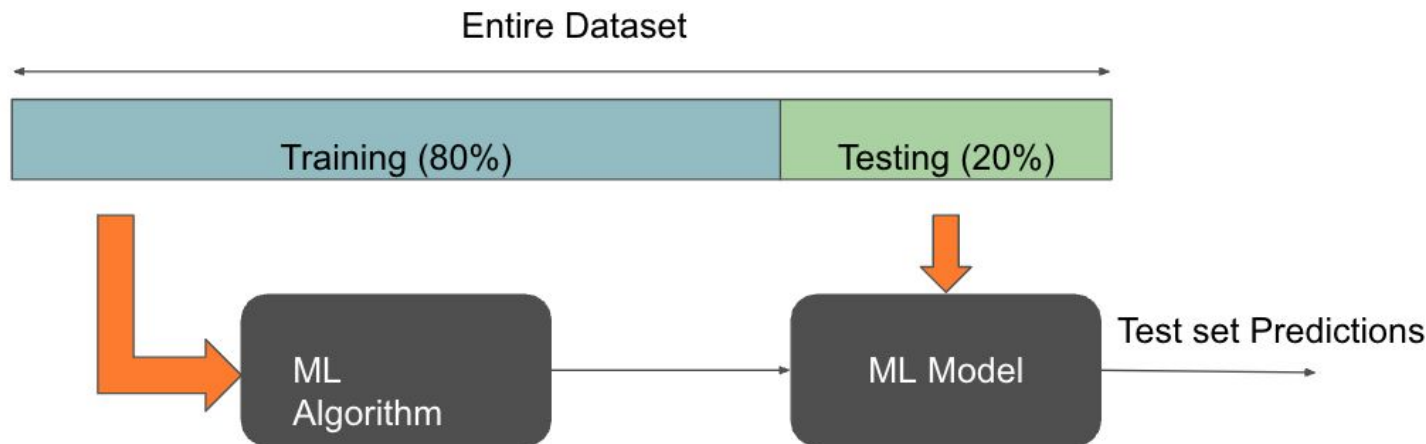
Over Fit
High Variance, Low Bias

Finding the right fit

How can we achieve a model with high precision and high accuracy?

Step number 1 is: What are my model's validation metrics?

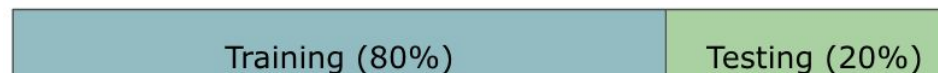
After training your model, you need to test it and evaluate it using appropriate metrics to understand where it falls in the accuracy-precision tradeoff.



Curating the perfect Testing Set

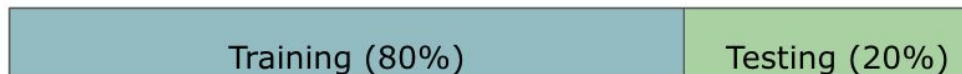
Previously, we tackled the problem of bias training sets. Testing sets can be biased as well. You need to make sure:

- The testing set is big enough for the metrics to be accurate
- The testing set includes all the possible classes/ranges/feature possibilities



50 Pass
30 Fail

0 Pass
20 Fail



40 Pass
40 Fail

10 Pass
10 Fail

Curating the perfect Testing Set

Whether your dataset is too large and complex or too small, selecting different random sub-samples for the testing dataset will yield different evaluation results.

So, how do we uncover our model's average performance?

We can use the K-fold method!



Curating the perfect Testing Set

We split our data into “K” folds. In our example $k=5$:

Initial Dataset

Test	Train	Train	Train	Train
Train	Test	Train	Train	Train
Train	Train	Test	Train	Train
Train	Train	Train	Test	Train
Train	Train	Train	Train	Test

Test 1 score

Test 2 score

Test 3 score

Test 4 score

Test 5 score

Average
Score

Evaluation of Statistical Supervised Learning Models

Classification Evaluation Metrics

Evaluation of Binary Classifiers

At the heart of many metrics is the confusion matrix. For a binary classifier, it looks like this:

$$Accuracy = \frac{TP + TN}{Total}$$

TP	FP	Total Predicted Positive
FN	TN	Total Predicted Negative
Total Real Positives	Total Real Negatives	

Evaluation of Binary Classifiers

While accuracy offers a good overview of the model's performance, we can infer some more detailed analysis on the model's performance.

True positive rate: **The proportion of actual positives that are correctly identified.**

A high TPR means that most of the positive cases are detected by the classifier.

$$TPR = \frac{TP}{\text{Total Real positives}}$$

TP	FP	Total Predicted Positive
FN	TN	Total Predicted Negative
Total Real Positives	Total Real Negatives	



Evaluation of Binary Classifiers

False positive rate: **The proportion of actual negatives that are incorrectly classified as positives.**

A low FPR indicates that the classifier is good at not misclassifying negatives as positives. It represents the rate of "false alarms."

$$FPR = \frac{FP}{Total\ Real\ Negatives}$$

TP	FP	Total Predicted Positive
FN	TN	Total Predicted Negative
Total Real Positives	Total Real Negatives	

Evaluation of Binary Classifiers

False negative rate: **The proportion of actual positives that are incorrectly classified as negatives.**

$$\text{FNR} = 1 - \text{TPR}$$

A low FNR is crucial when missing a positive case is particularly costly (for instance, in medical diagnostics).

$$\text{FNR} = \frac{\text{FN}}{\text{Total Real Positives}}$$

TP	FP	Total Predicted Positive
FN	TN	Total Predicted Negative
Total Real Positives	Total Real Negatives	

Evaluation of Binary Classifiers

True Negative rate: **The proportion of actual negatives that are correctly identified.**

A high TNR means that the classifier is effective at correctly recognizing negatives.

$$TNR = \frac{TN}{Total\ Real\ negatives}$$

TP	FP	Total Predicted Positive
FN	TN	Total Predicted Negative
Total Real Positives	Total Real Negatives	

Evaluation of Binary Classifiers

Precision: It measures the proportion of positive identifications that were actually correct.

High precision indicates that the model has a low rate of false positives.

$$Precision = \frac{TP}{Total\ Predicted\ Positive}$$

TP	FP	Total Predicted Positive
FN	TN	Total Predicted Negative
Total Real Positives	Total Real Negatives	

Evaluation of Binary Classifiers

Recall: It measures the proportion of actual positives that were correctly identified.

High recall means that most of the positive instances are captured by the model.

$$\text{Recall} = \frac{TP}{\text{Total Number of Positives}}$$

TP	FP	Total Predicted Positive
FN	TN	Total Predicted Negative
Total Real Positives	Total Real Negatives	

Evaluation of Binary Classifiers

F1 Score: When you need a balance between precision and recall, especially under uneven class distributions.

A higher F1 score indicates a better balance between the precision and recall of the model.

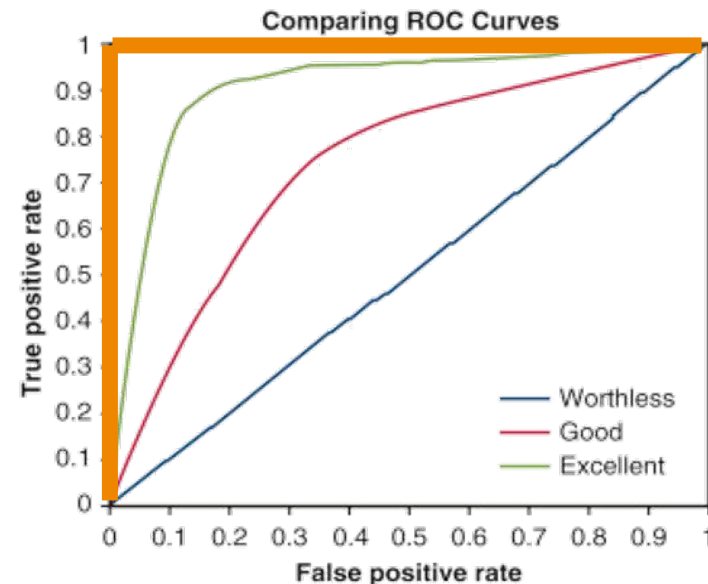
$$F1\ score = \frac{2\ Precision \times Recall}{Precision + Recall}$$

TP	FP	Total Predicted Positive
FN	TN	Total Predicted Negative
Total Real Positives	Total Real Negatives	

Evaluation of Binary Classifiers

The ROC curve is a graphical plot that illustrates the diagnostic ability of a binary classifier as its discrimination threshold is varied. It plots the True Positive Rate (TPR) (i.e., recall) against the False Positive Rate (FPR)

AUC measures the overall performance across all classification thresholds. An AUC of 0.5 suggests no discriminative power (equivalent to random guessing), while an AUC of 1.0 indicates perfect classification.

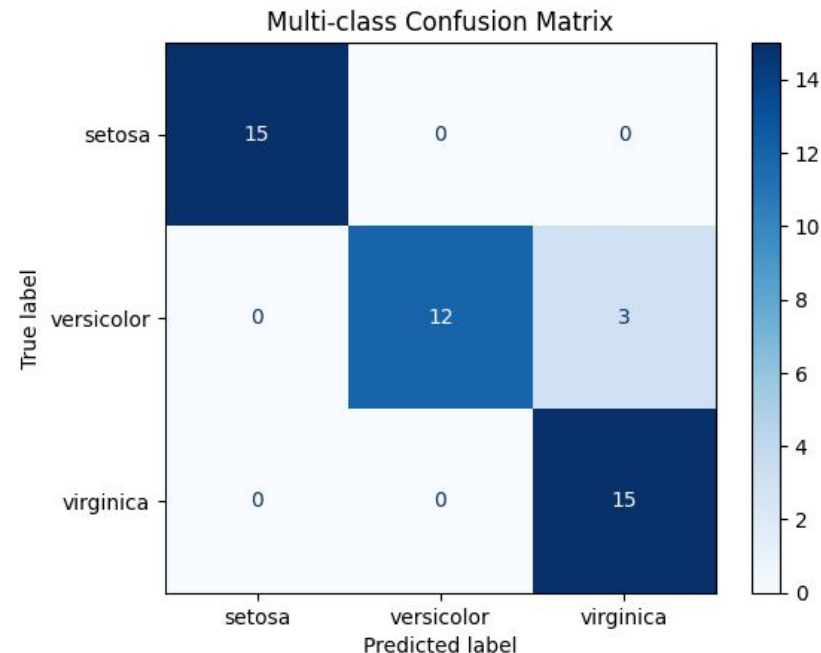


Evaluation of multi-class classification

Instead of a 2×2 matrix, a multi-class confusion matrix is an $N \times N$ matrix (where N is the number of classes). Each cell C_{ij} represents the number of instances of true class i that were predicted as class j .

The diagonal cells show the correct predictions, while the off-diagonals indicate the types of misclassifications between classes.

Example using Iris dataset:

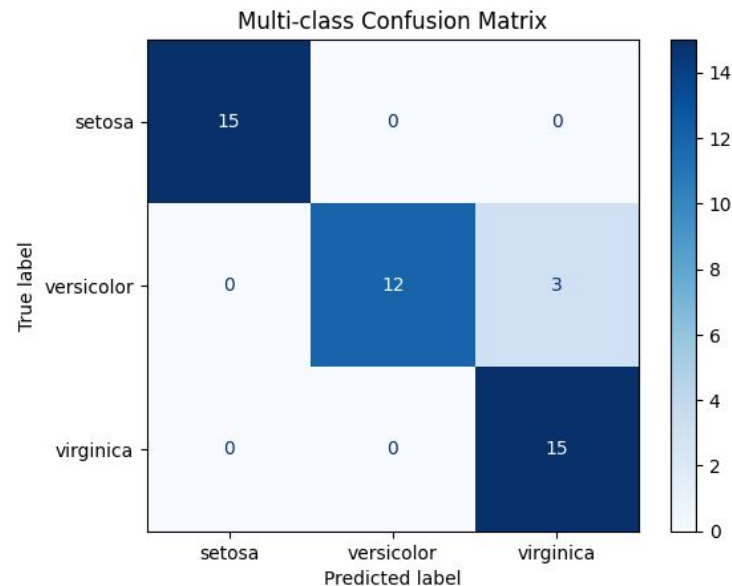


Evaluation of multi-class classification

The same metrics discussed above are calculated per class.

Precision: Measures how many of the instances predicted as class i were actually class i .

$$\text{Precision}_i = \frac{C_{ii}}{\sum_{j=1}^N C_{ji}}$$

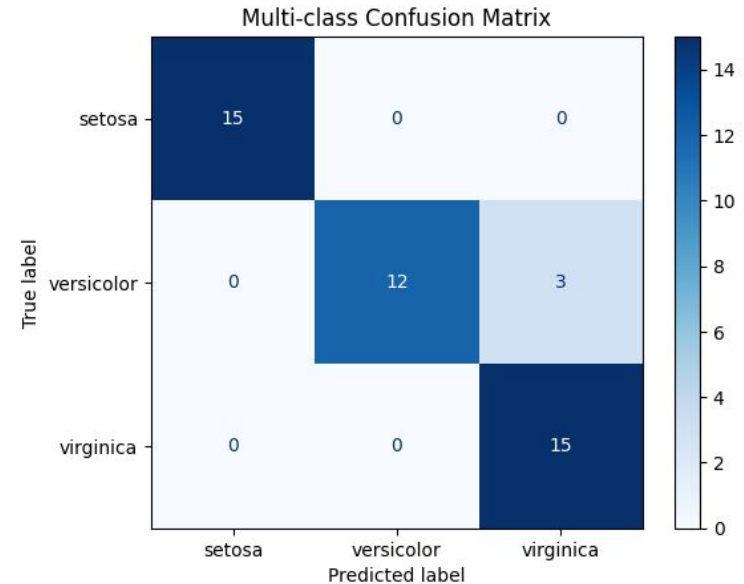


Evaluation of multi-class classification

The same metrics discussed above are calculated per class.

Recall: Evaluates how many of the actual class i instances were correctly identified.

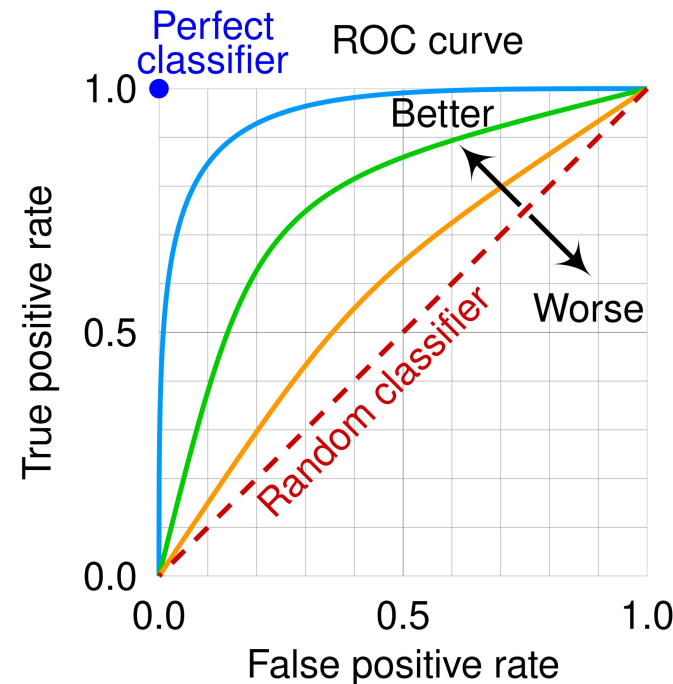
$$\text{Recall}_i = \frac{C_{ii}}{\sum_{j=1}^N C_{ij}}$$



Evaluation of multi-class classification

Area Under the ROC Curve (AUC) for Multi-Class:

Though more common in binary settings, you can compute AUC for each class using a one-vs-rest approach or use macro/micro averaging across classes. However, interpretation can be more complex in a multi-class context.



Regression Evaluation Metrics

Evaluation of Regression Metrics

When evaluating regression models, the goal is to measure how well the predicted continuous outcomes match the actual values.

In regression, the evaluation metrics used can be the same as the error metrics covered in the previous chapters:

$$\begin{aligned} MBE &= \frac{1}{n} \sum_{i=1}^n (y_i - y'_i) & RMSE &= \sqrt{\frac{\sum_{i=1}^n (\hat{y}_i - y_i)^2}{n}} \\ MAE &= \frac{1}{n} \sum_{i=1}^n |y_i - y'_i| & MSE &= \frac{1}{n} \sum_{i=1}^n (y_i - y'_i)^2 \end{aligned}$$

Evaluation of SML Algorithms in Section 1 of lab 'Evaluation of Supervised Learning Algorithms'

Evaluation of Unsupervised Machine Learning Algorithms

Evaluation of Unsupervised Algorithms

When evaluating unsupervised learning, the challenge is that there's typically no "ground truth" to compare against.

Therefore, we use specialized metrics that measure the quality of the discovered structures or relationships.

How to evaluate unsupervised learning effectively?



Ground Truth Comparison

Traditional evaluation
methods



Specialized Metrics

Adapted for unsupervised
learning

Evaluation of clustering

Clustering Evaluation Metrics

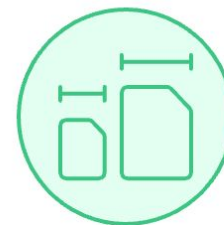
Clustering evaluation metrics are generally divided into internal and external measures.

Choose the appropriate clustering evaluation metrics for analysis



Internal Measures

Assess cohesion and separation within the data



External Measures

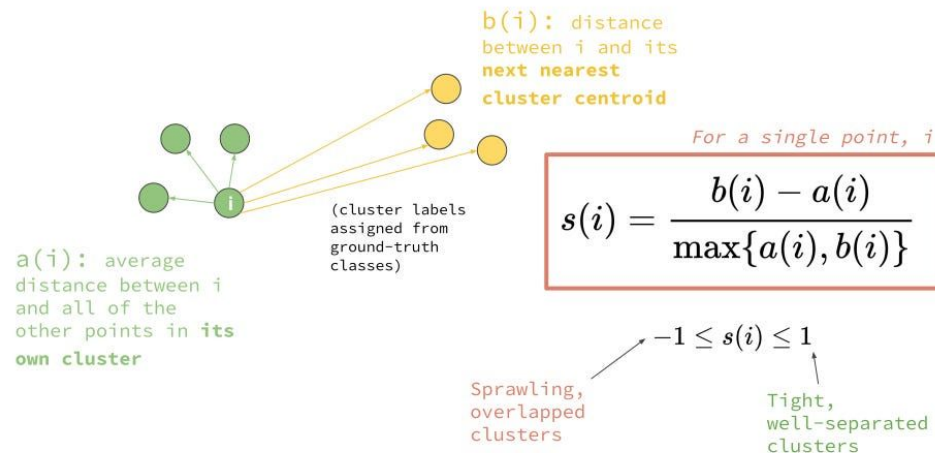
Compare with external benchmarks or ground truth

Clustering Evaluation Metrics - Internal Metrics

These metrics evaluate the quality of the clusters based solely on the data that was clustered.

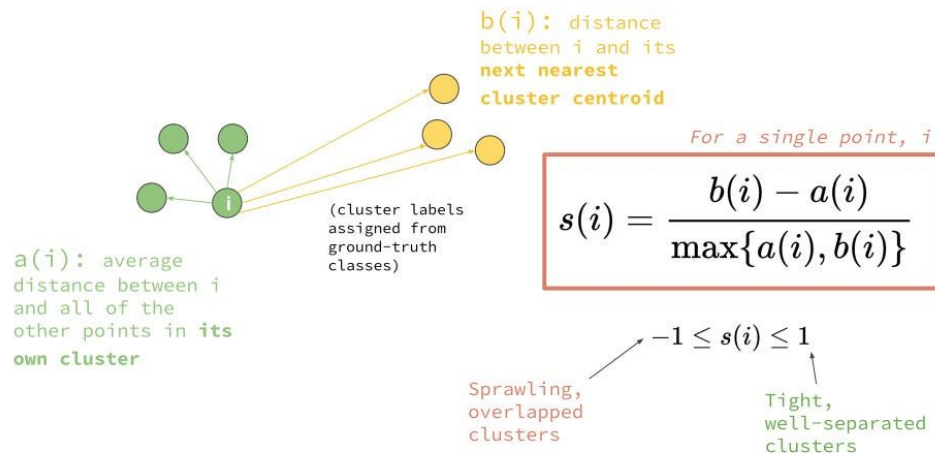
1. Silhouette Score: Measures the degree of separation between clusters.

How It's Computed: For each data point, compute the difference between the average distance to points in the same cluster (cohesion) and the average distance to points in the nearest neighboring cluster (separation), then normalize this difference.



Clustering Evaluation Metrics - Internal Metrics

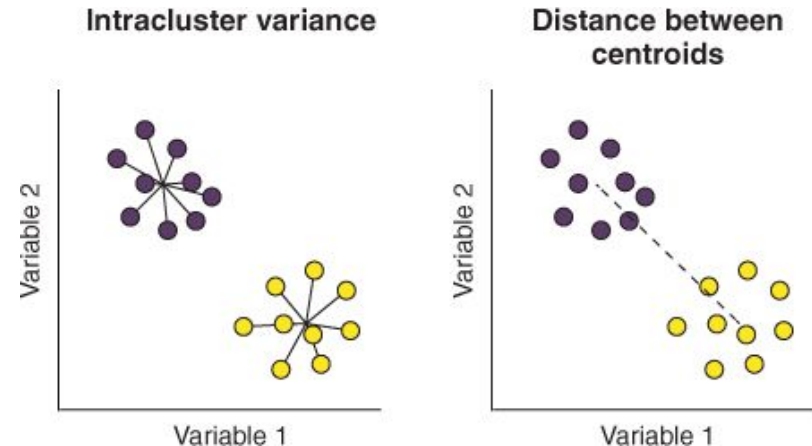
Scores range from -1 to 1. A score near +1 indicates that the data point is well matched to its own cluster and poorly matched to neighboring clusters; a score near 0 indicates overlapping clusters; and negative values indicate that the point may have been assigned to the wrong cluster.



Clustering Evaluation Metrics - Internal Metrics

2. **Davies-Bouldin Index (DBI):** It is calculated as the average similarity measure of each cluster with the cluster most similar to it. In this context, similarity is defined as the ratio between inter-cluster and intra-cluster distances. As such, this index ranks well-separated clusters with less dispersion as having a better score.

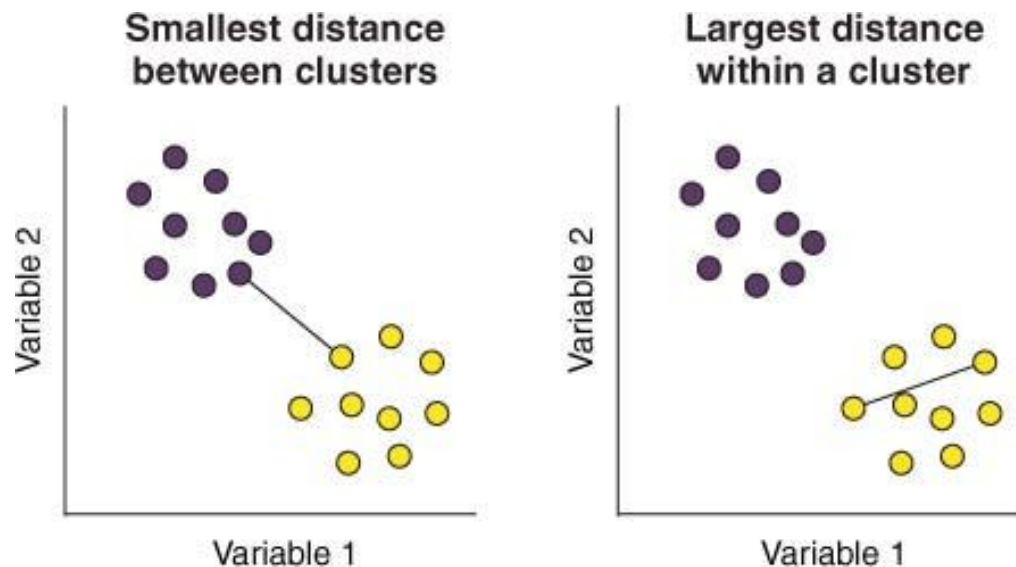
Interpretation: Lower values indicate better clustering because clusters are compact and distinct from each other.



Clustering Evaluation Metrics - Internal Metrics

3. **Dunn Index:** The ratio between the smallest distance between observations not in the same cluster (inter-cluster distance) and the largest intra-cluster distance.

Interpretation: A higher Dunn index indicates better clustering with well-separated clusters.



Clustering Evaluation Metrics - External Metrics

When true labels or a reference clustering is available, external metrics can quantify how well the clustering algorithm recovered the known structure.

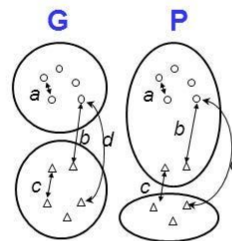
1. Adjusted Rand Index (ARI):

Similarity between the predicted clusters and the true labels, adjusted for chance.

Interpretation: Scores range from -1 to 1, with 1 indicating perfect agreement.

Rand and Adjusted Rand index

[Rand, 1971] [Hubert and Arabie, 1985]



Agreement: a, d

Disagreement: b, c

$$RI(P, G) = \frac{a + d}{a + b + c + d}$$

$$ARI = \frac{RI - E(RI)}{1 - E(RI)}$$

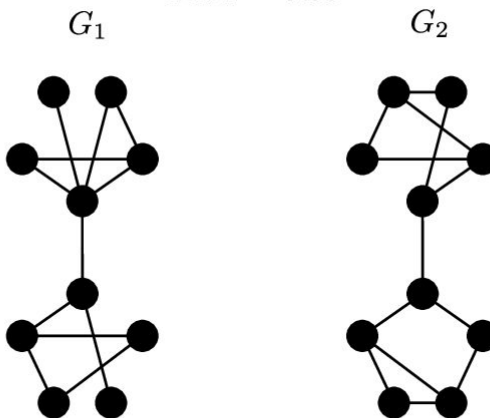
Clustering Evaluation Metrics - External Metrics

2. Normalized Mutual Information (NMI): The mutual dependence between the true labels and the predicted clusters, normalized to range between 0 and 1.

Interpretation: Higher values indicate better alignment between the clusters and the true classes.

(a)

$NMI = 0.06$



$$E_1 = 12, E_2 = 13, E_{12} = 6$$

Evaluation of Market Basket Analysis

MBA Evaluation Metrics

Market basket analysis focuses on discovering associations and co-occurrences between items in transactional data. The evaluation of association rules relies on metrics that assess the strength, reliability, and interestingness of the rules.

In addition to the support, confidence, and lift that were introduced previously, we can also calculate the **leverage** and **conviction**.



Rule: $X \Rightarrow Y$

- $Support = \frac{Frequency(X, Y)}{N}$
- $Confidence = \frac{Frequency(X, Y)}{Frequency(X)}$
- $Lift = \frac{Support}{Support(X) \times Support(Y)}$

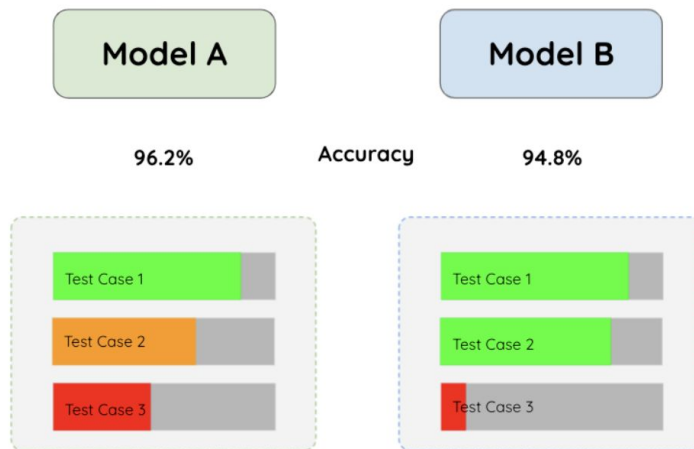
Evaluation of UML algorithms lab

Comparing ML Models

Why not always choose the most powerful one?



- There's no most powerful model in general.
- It's all data related:
 - Sometimes the most powerful models perform poorly on your data, where some simpler ones will do a better job.
 - This is why we have to try different settings and see what works best.





Comparing Apples to Apples

When comparing 2 models

- You should be using the same dataset
- Models should be trained on the same training set
- Models should be evaluated on the same test set

Otherwise, it will be an **unfair** comparison.

Statistical Hypothesis Tests



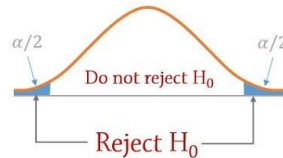
- Start with an initial hypothesis (*assumption*)
- Propose an alternative hypothesis
- Conduct the test
- To reject or not to reject the initial hypothesis

Hypothesis Testing

Two-tailed

$$H_0: \mu = 23$$

$$H_1: \mu \neq 23$$

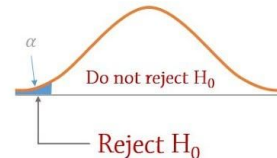


One-tailed

Left-tailed Right-tailed

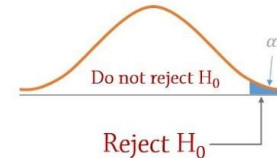
$$H_0: \mu \geq 23$$

$$H_1: \mu < 23$$

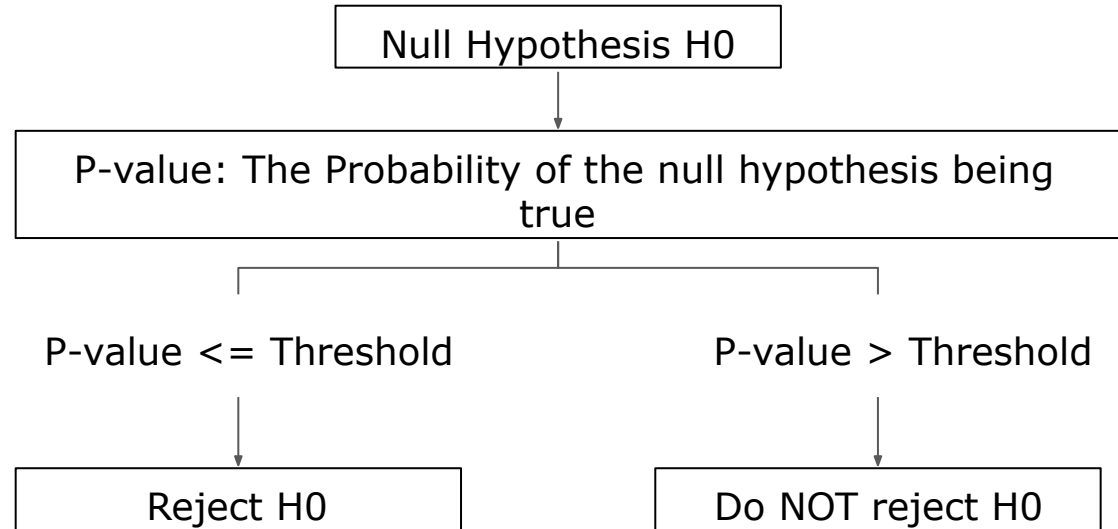


$$H_0: \mu \leq 23$$

$$H_1: \mu > 23$$



Statistical Hypothesis Tests



Statistical Hypothesis Tests

- In ML, to compare models, we use statistical tests.
- **The null hypothesis is that the models perform the same.**
- We should have an alternate hypothesis that complements the null one (e.g. model A is better than model B).
- If we reject the null hypothesis, we reject it in favor of the alternative hypothesis.

What is the Null Hypothesis (H_0)

In statistics (and by extension in ML), the null hypothesis is the idea that nothing special is happening.

It's a default assumption that there's no effect, no difference, or no relationship between variables.

Example: Let's say you're testing if a new model performs better than the old one.

- H_0 (null hypothesis): The new model performs the same as the old one (no improvement).
- H_1 (alternative hypothesis): The new model performs better than the old one.

You start by assuming the null hypothesis is true until you find enough evidence (data) to reject it.

Why it matters in ML

In ML, you often compare models or evaluate features.

For example:

- Is Model B's accuracy truly higher than Model A's, or is it just random noise?
- Does a specific feature really improve prediction, or not?

Hypothesis testing helps you avoid **overclaiming**.

You don't want to say "the model improved" when, statistically, it might not have.

Z-Score test

What is a Z-score?

The Z-score tells you how far a data point is from the average, measured in units of standard deviation.

Formula:

$$Z = \frac{X - \mu}{\sigma}$$

where:

- X = your observed value
- μ = mean of the data
- σ = standard deviation

What is a Z-score?

Example:

If your model accuracy is 0.82, the average accuracy of similar models is 0.75, and the standard deviation is 0.05:

$$Z = \frac{0.82 - 0.75}{0.05} = 1.4$$

That means your result is 1.4 standard deviations above the mean.

How Z-score connects to hypothesis testing

The Z-score helps you decide how unlikely your result is under the null hypothesis.

If the Z-score is very large (say, greater than 1.96 or less than -1.96), your result is in the rare part of the distribution (only 5% of cases).

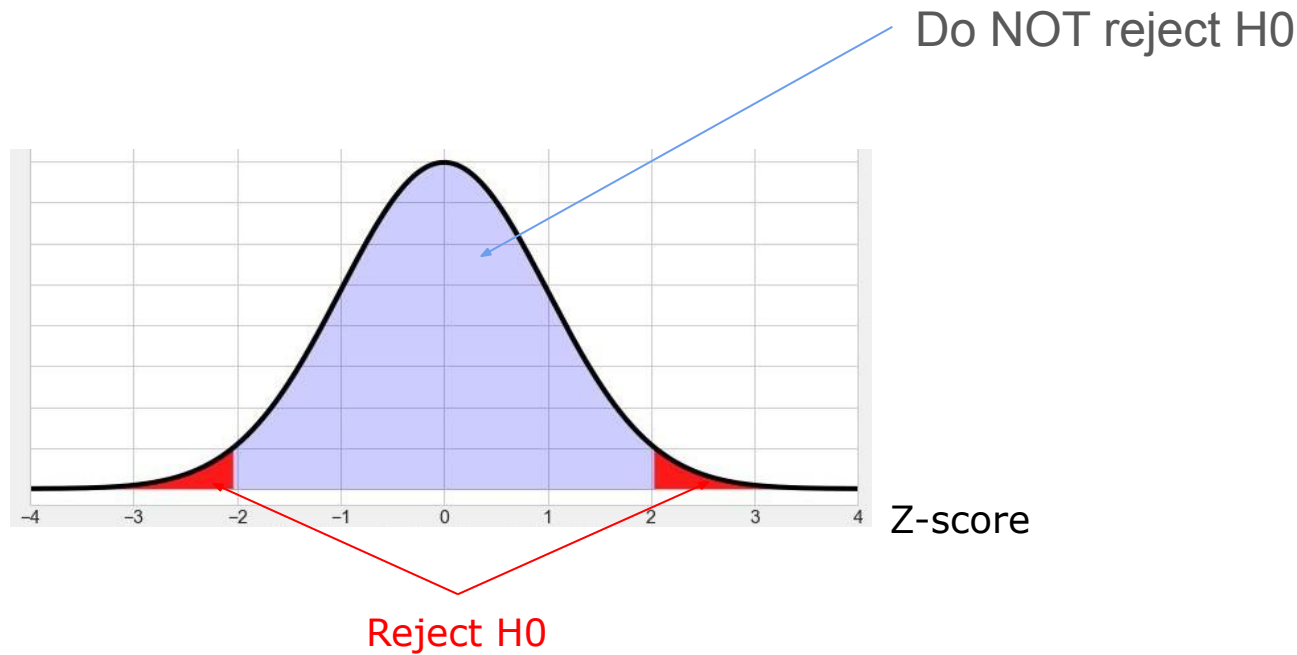
That means it's unlikely to happen by chance → you reject the null hypothesis.

In ML terms: your model's improvement is statistically significant.



Testing Difference in Proportion (Z-Score)

- The Z-Score uses the Gaussian Distribution.



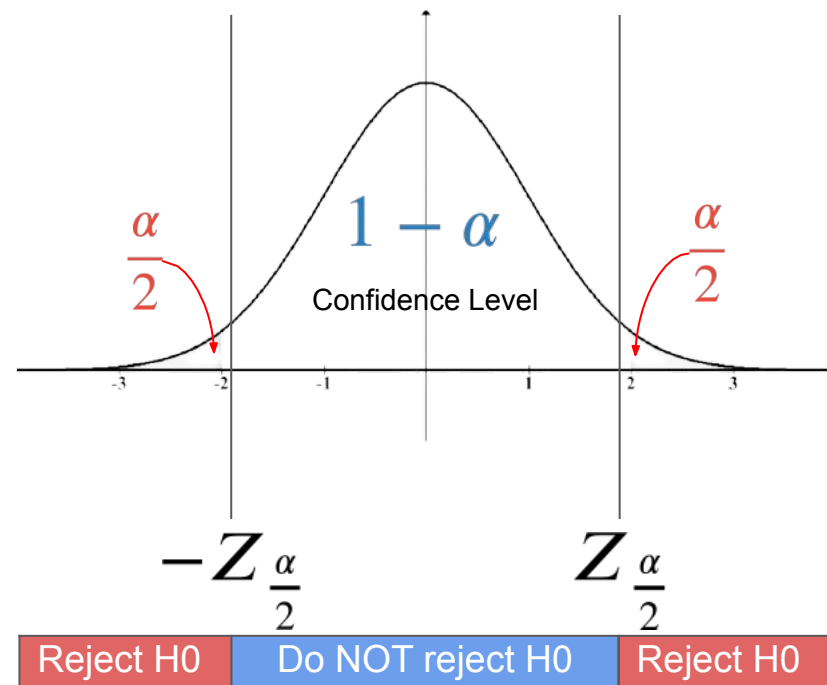


Testing Difference in Proportion (Z-Score)

We can calculate the Z-value instead of the p-value to know if we should reject H_0 or not.

Confidence Level	α (level of significance)	$Z_{\alpha/2}$
99%	1%	2.575
95%	5%	1.96
90%	10%	1.645

<https://s3-us-west-2.amazonaws.com/courses-images/wp-content/uploads/sites/1888/2017/05/11170656/3159.png>



Testing Difference in Proportion (Z-Score)

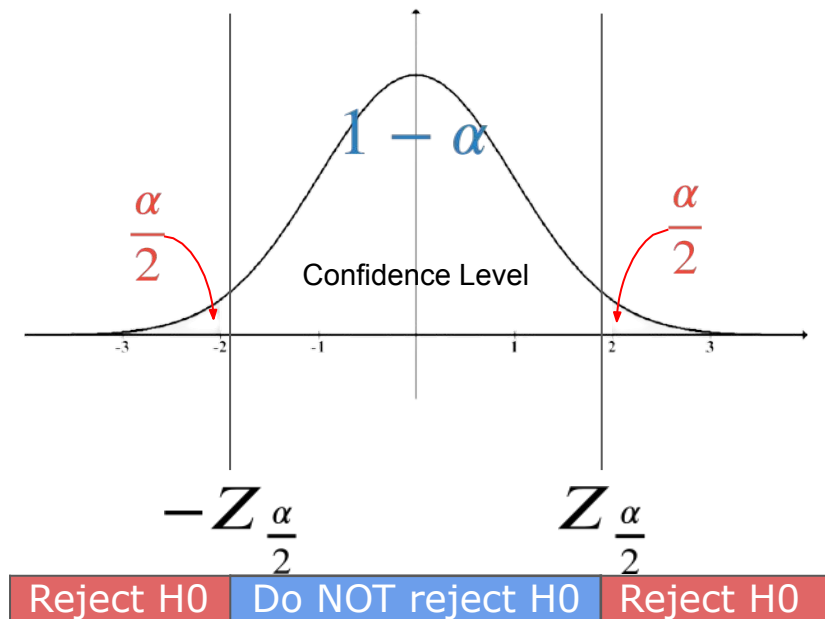
Let's say we are comparing two models:

$$\sigma^2 = \frac{ACC(1 - ACC)}{n} \quad \leftarrow \text{Size of the Test Set}$$

$$z = \frac{(ACC_1 - ACC_2)}{\sqrt{\sigma_1^2 + \sigma_2^2}}$$

If **z** falls in the rejection regions, we reject the null hypothesis, **otherwise**, we do not reject the null hypothesis.

Recall: Null hypothesis is that model 1 and model 2 perform the same.



Testing Difference in Proportion (Z-Score)



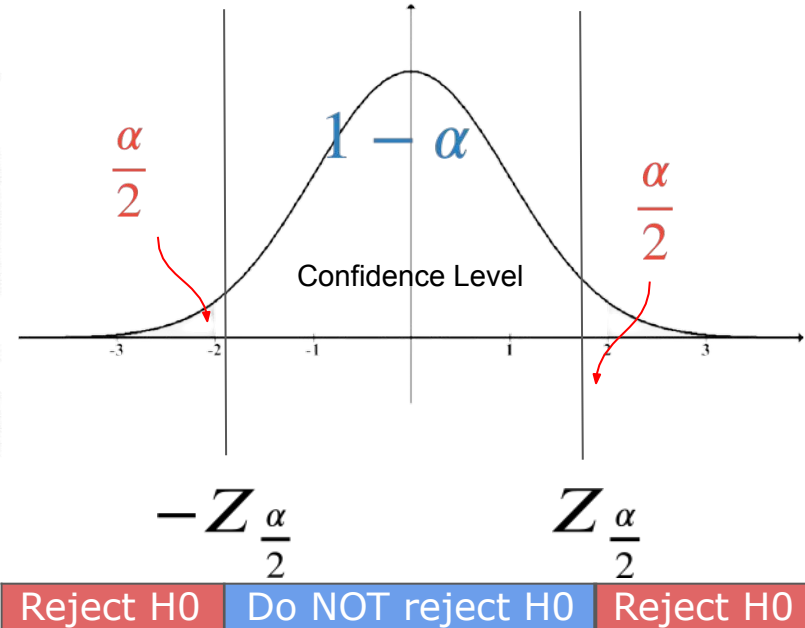
Cumulative Standard Normal Distribution (i.e., $P(Z > z)$, the area to the right of the cut off)

Z	.00	.01	.02	.03	.04	.05	.06	.07	.08	.09
0.0	0.5000	0.4960	0.4920	0.4880	0.4840	0.4801	0.4761	0.4721	0.4681	0.4641
0.1	0.4602	0.4562	0.4522	0.4483	0.4443	0.4404	0.4364	0.4325	0.4286	0.4247
0.2	0.4207	0.4168	0.4129	0.4090	0.4052	0.4013	0.3974	0.3936	0.3897	0.3859
0.3	0.3821	0.3783	0.3745	0.3707	0.3669	0.3632	0.3594	0.3557	0.3520	0.3483
0.4	0.3446	0.3409	0.3372	0.3336	0.3300	0.3264	0.3228	0.3192	0.3156	0.3121
0.5	0.3085	0.3050	0.3015	0.2981	0.2946	0.2912	0.2877	0.2843	0.2810	0.2776
1.5	0.0668	0.0655	0.0643	0.0630	0.0618	0.0606	0.0594	0.0582	0.0571	0.0559
1.6	0.0548	0.0537	0.0526	0.0516	0.0505	0.0495	0.0485	0.0475	0.0465	0.045
1.7	0.0446	0.0436	0.0427	0.0418	0.0409	0.0401	0.0392	0.0384	0.0375	0.0367
1.8	0.0359	0.0351	0.0344	0.0336	0.0329	0.0322	0.0314	0.0307	0.0301	0.0294
1.9	0.0287	0.0281	0.0274	0.0268	0.0262	0.0256	0.0250	0.0244	0.0239	0.0233

<https://sphweb.bumc.bu.edu/otlt/mph-modules/bs/sas/sas4-onesampltest/Table1.PNG>

Z-values

p-values



What the p-value means

The p-value tells you the probability of seeing a result at least as extreme as yours if the null hypothesis were true.

In simple terms: The smaller the p-value, the less likely your result is due to random chance.

Typical thresholds:

- $p < 0.05 \rightarrow$ statistically significant (you can reject H_0)
- $p < 0.01 \rightarrow$ very strong evidence against H_0
- $p > 0.05 \rightarrow$ not significant (could easily happen by chance)

McNemar Test

McNemar Test



AMERICAN
UNIVERSITY
OF BEIRUT

The McNemar Test is applied on a 2x2 confusion matrix

$$Acc_1 = \frac{A + B}{A + B + C + D}$$

$$Acc_2 = \frac{A + C}{A + B + C + D}$$

	Model 2 Correct	Model 2 Wrong
Model 1 Correct	A	B
Model 1 Wrong	C	D

McNemar Test

- The null hypothesis is that both models perform the same.
- We compute the Chi-squared value:

$$\chi^2 = \frac{(B - C)^2}{B + C} \longrightarrow \text{Chi Squared Probability table} \longrightarrow p - value$$

- We then compare this p-value to our threshold alpha to see if we reject the null hypothesis or not.

McNemar Test



Suppose we have the 2 models represented by this confusion matrix.

$$Acc_1 = \frac{A + B}{A + B + C + D} = \frac{9970}{10000} = 99.7\%$$

$$Acc_2 = \frac{A + C}{A + B + C + D} = \frac{9960}{10000} = 99.6\%$$

		Model 2 Correct	Model 2 Wrong
Model 1	Correct	A 9959	B 1
	Wrong	C 1	D 29

McNemar Test



Suppose we have the 2 models represented by this confusion matrix.

H0: The 2 models perform the same way

Significance Threshold: alpha = 0.05

$$\chi^2 = \frac{(B - C)^2}{B + C} = \frac{10^2}{12} = 8.3$$

D	Upper one-sided α								
	.975	.95	.50	.25	.10	.05	.025	.01	.001
1	.001	.004	.455	1.32	2.71	3.84	5.02	6.63	10.83



Location of our chi2 value

$p - value = (0.01, 0.001) < 0.05 \rightarrow$ We **reject** the null hypothesis

		Model 2	
		Correct	Wrong
Model 1	Correct	A 9959	B 1
	Wrong	C 1	D 29

McNemar Test

How to interpret this result?

- This test relies on the value of χ^2 , which is dependent on B and C.
- Model 1 predicted 11 examples correctly that model 2 did not.
- Model 2 predicted 1 example correctly that model 1 did not.
- This is why it is intuitive to deduce that model 1 is better than 2.

		Model 2 Correct	Model 2 Wrong
Model 1 Correct	A 9959	B 1	
Model 1 Wrong	C 1	D 29	

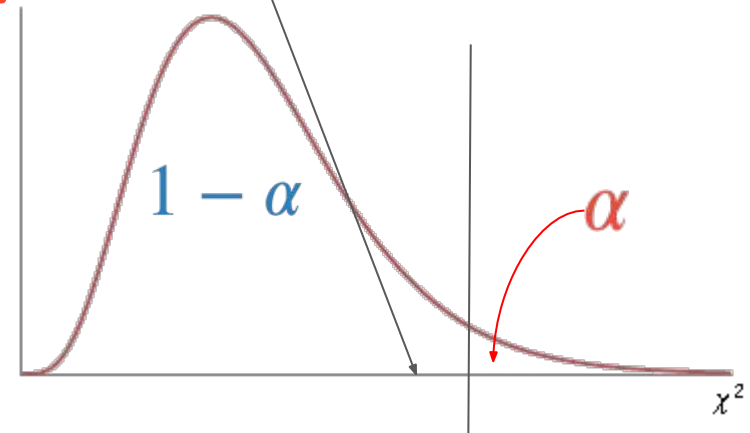
McNemar Test



p - values

χ^2 values

DF	0.995	0.975	0.2	0.1	0.05	0.025	0.02	0.01	0.005	0.002	0.001
1	.0004	.00016	1.642	2.706	3.841	5.024	5.412	6.635	7.879	9.55	10.828
2	0.01	0.0506	3.219	4.605	5.991	7.378	7.824	9.21	10.597	12.429	13.816
3	0.0717	0.216	4.642	6.251	7.815	9.348	9.837	11.345	12.838	14.796	16.266
4	0.207	0.484	5.989	7.779	9.488	11.143	11.668	13.277	14.86	16.924	18.467
5	0.412	0.831	7.289	9.236	11.07	12.833	13.388	15.086	16.75	18.907	20.515
6	0.676	1.237	8.558	10.645	12.592	14.449	15.033	16.812	18.548	20.791	22.458
7	0.989	1.69	9.803	12.017	14.067	16.013	16.622	18.475	20.278	22.601	24.322
8	1.344	2.18	11.03	13.362	15.507	17.535	18.168	20.09	21.955	24.352	26.124
9	1.735	2.7	12.242	14.684	16.919	19.023	19.679	21.666	23.589	26.056	27.877
10	2.156	3.247	13.442	15.987	18.307	20.483	21.161	23.209	25.188	27.722	29.588
11	2.603	3.816	14.631	17.275	19.675	21.92	22.618	24.725	26.757	29.354	31.264
12	3.074	4.404	15.812	18.549	21.026	23.337	24.054	26.217	28.3	30.957	32.909
13	3.565	5.009	16.985	19.812	22.362	24.736	25.472	27.688	29.819	32.535	34.528
14	4.075	5.629	18.151	21.064	23.685	26.119	26.873	29.141	31.319	34.091	36.123
15	4.601	6.262	19.311	22.307	24.996	27.488	28.259	30.578	32.801	35.628	37.697
16	5.142	6.908	20.465	23.542	26.296	28.845	29.633	32	34.267	37.146	39.252
17	5.697	7.564	21.615	24.769	27.587	30.191	30.995	33.409	35.718	38.648	40.79
18	6.265	8.231	22.76	25.989	28.869	31.526	32.346	34.805	37.156	40.136	42.312
19	6.844	8.907	23.9	27.204	30.144	32.852	33.687	36.191	38.582	41.61	43.82
20	7.434	9.591	25.038	28.412	31.41	34.17	35.02	37.566	39.997	43.072	45.315



Do NOT reject H0 Reject H0

Degrees of Freedom

Comparing ML Models lab in ‘Evaluation of Supervised Learning Algorithms’

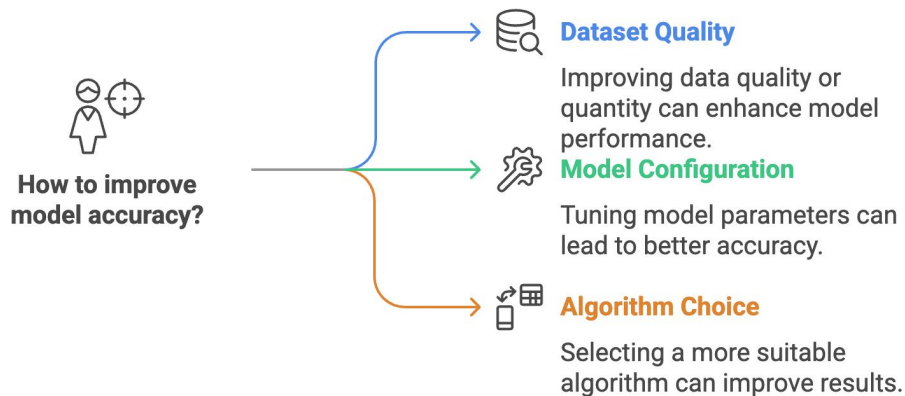
Section 2

Model Tuning

When do we tune the ML Model

After you've thoroughly tested your ML models, you might realize that the performance is sub-optimal, and you would want to increase the accuracy of your model.

Your model's shortcoming might be because of the dataset, model configuration, or choice of algorithm.



Using Data Methods to Improve Model Performance

Improving ML Performance using Data

If your problem is complex enough, you might need to gather more cases to expose to the model.

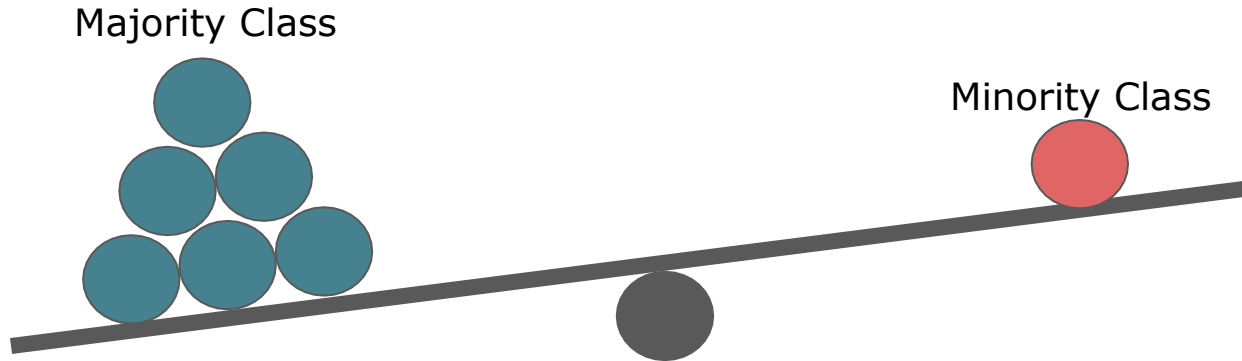
- ML requires data to be able to give accurate models.
- Getting more data can help you solve overfitting or even data imbalance.



Reshuffling in case of Data Imbalance

If you realize that one of the features (or the output feature) of your dataset is imbalance, you might consider:

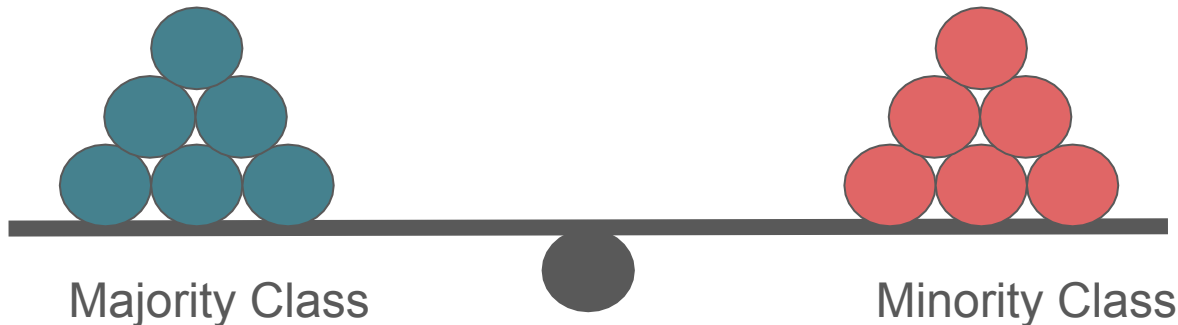
- Oversampling the Minority Class
- Undersampling the Majority Class



Reshuffling in case of Data Imbalance

Oversampling the Minority Class:

- We replicate randomly some examples from the minority class to have the same number as the majority class.
- We don't throw away information, but we have the same information several times which may lead to overfitting.

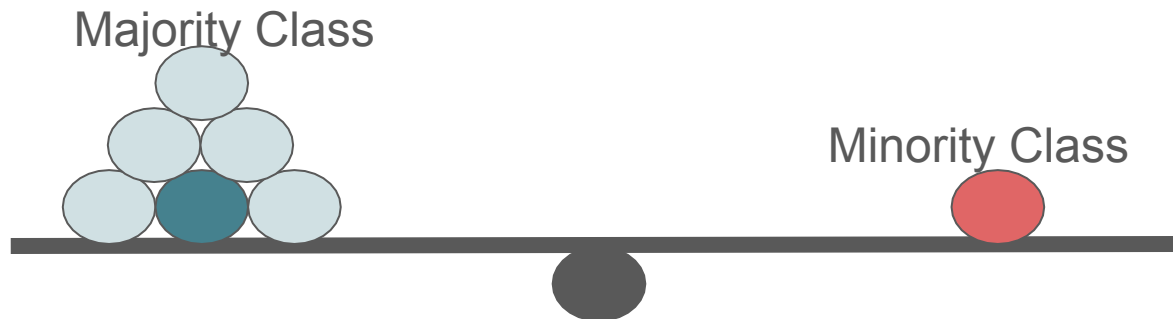


Reshuffling in case of Data Imbalance



Undersampling the Majority Class:

- We randomly take from the majority class the same number of examples as the minority class.
- This improves runtime of the model, but throws away useful information.



Creating More Features

Use existing features to create new ones when necessary

Initial Dataset

X1	Y



$$X_2 = X_1^2$$

X1	X2	Y

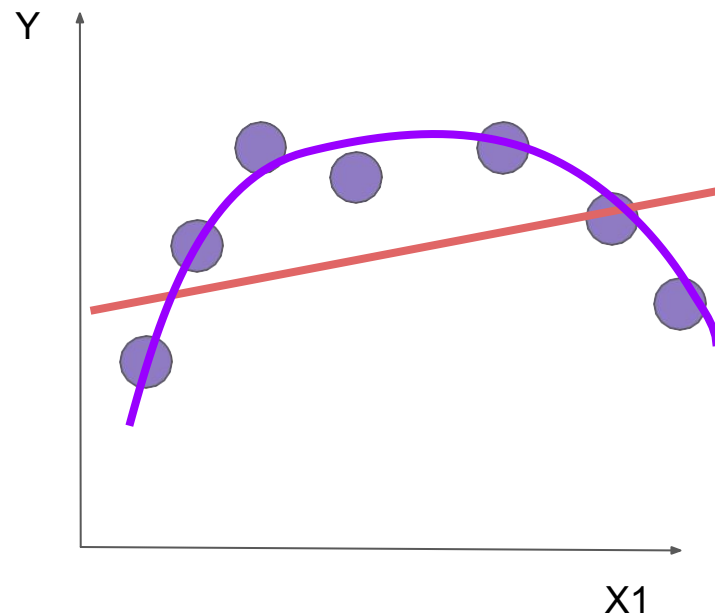
Linear Regression

$$y = a_0 + a_1 X_1$$

Multi-Linear Regression

$$y = a_0 + a_1 X_1 + a_2 X_2$$

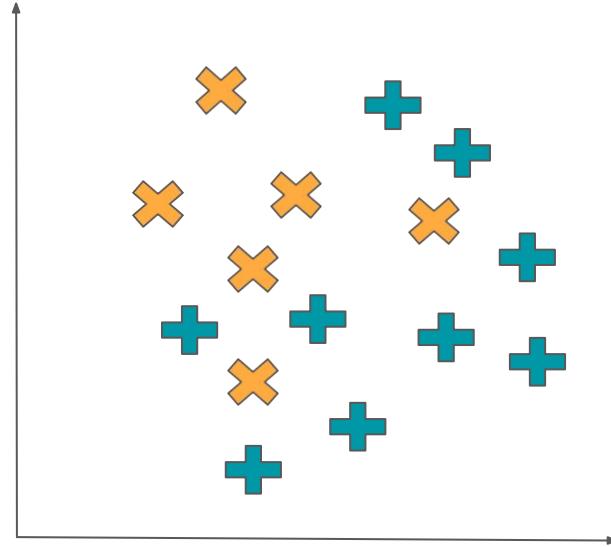
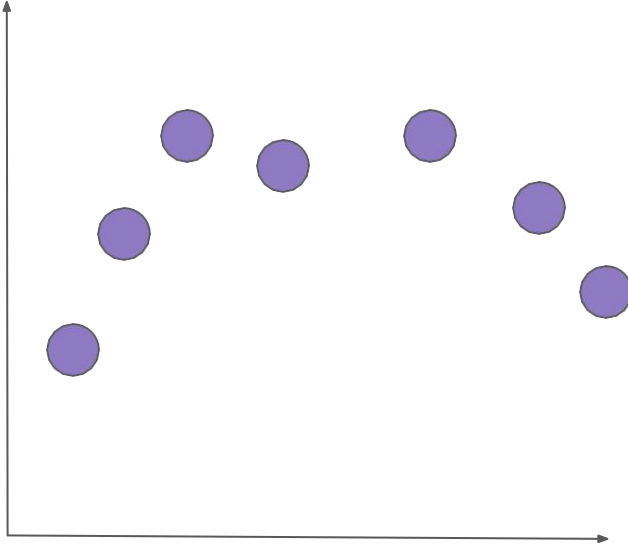
$$y = a_0 + a_1 X_1 + a_2 X_1^2$$



Using Model Design Methods to Improve Performance

Model Design

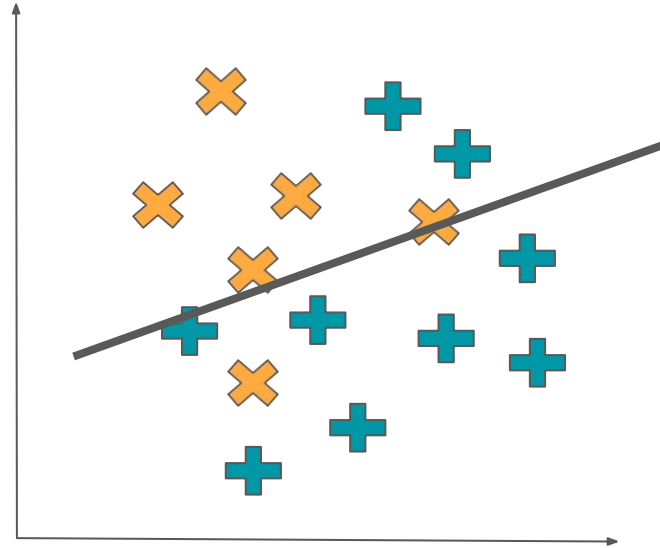
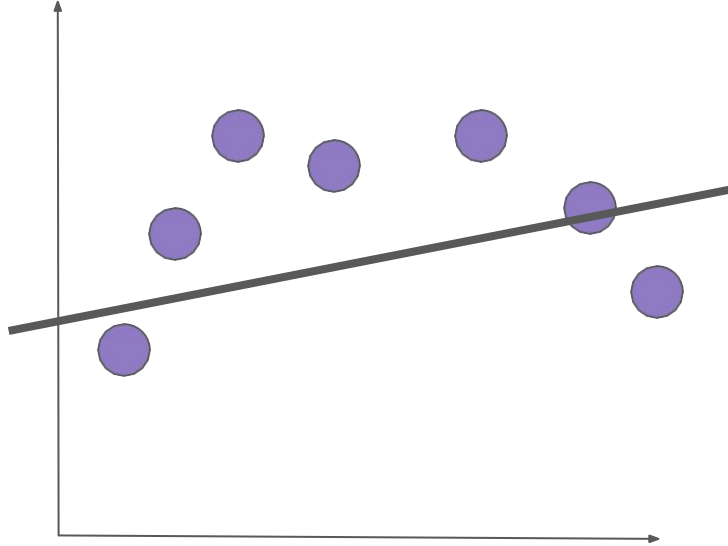
After finishing with data preparation, you need to build models that can make good use of the data.



Model Design

We first see that linear models do not make good predictions on our data.

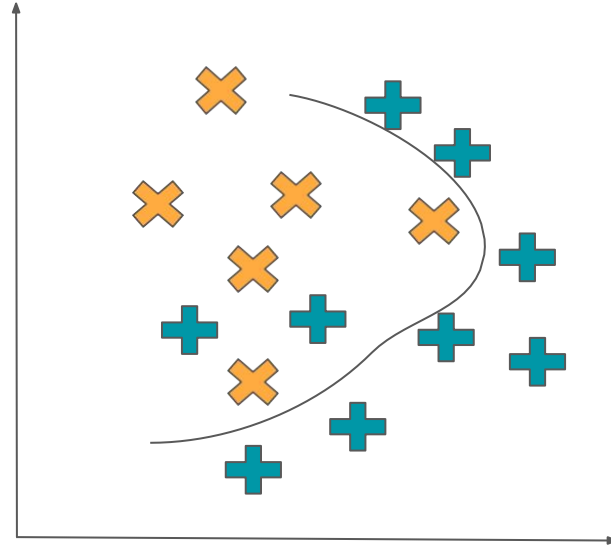
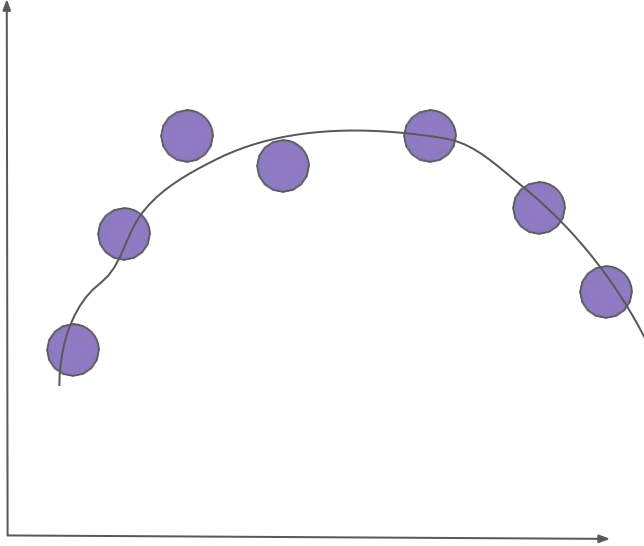
Degree 1 polynomials



Model Design

We try to go to nonlinear models to get better results.

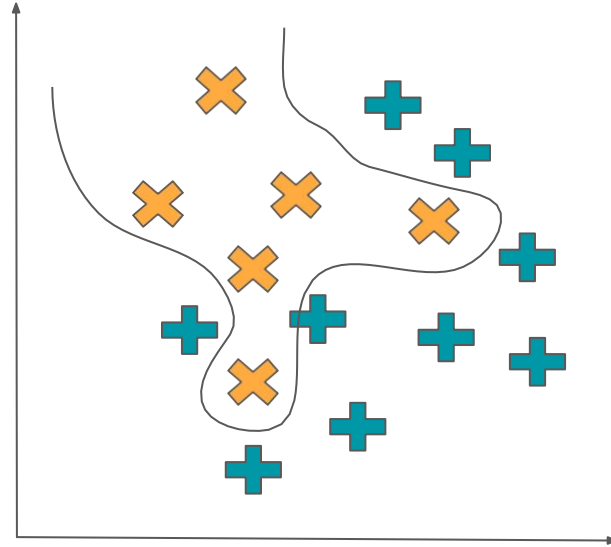
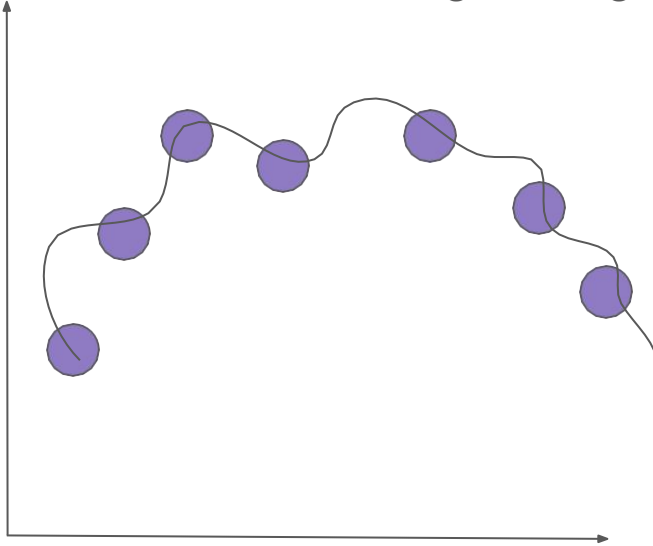
Degree 2 polynomials



Model Design

One might try to more complex models like the below, but this may cause **OVERFITTING**.

Higher degree polynomials



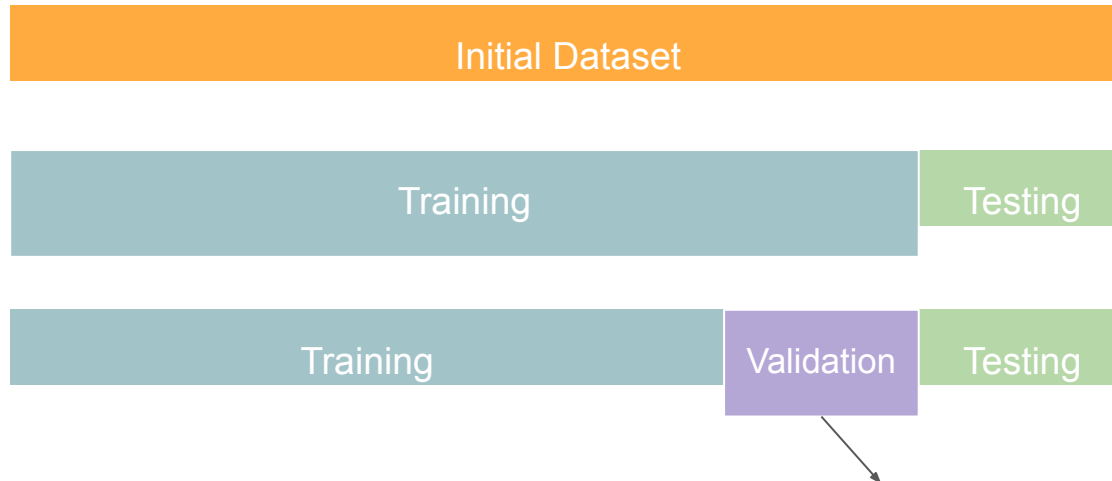
Parameter vs Hyperparameter

- A **Parameter** are the coefficients learned through the training process. It's a property of the model (the coefficients that a model learns.)
- A **Hyperparameter** is a parameter that controls the training process. It's a property of the learning algorithm (e.g. the learning rate.)

Thus we can try different hyperparameters to test their effect on model performance

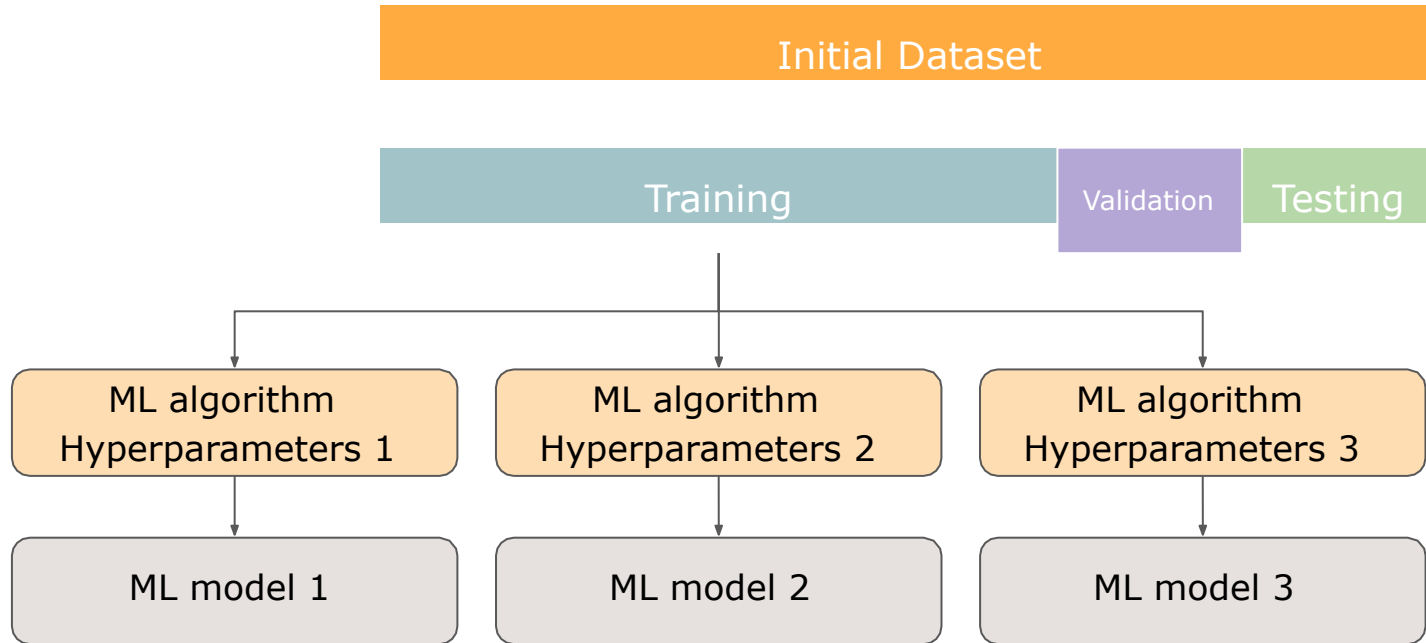
Hyperparameter Tuning

Instead of having 2 sets training and testing, we add another set for cross validation. We use this set to change and evaluate different hyperparameters of our model.

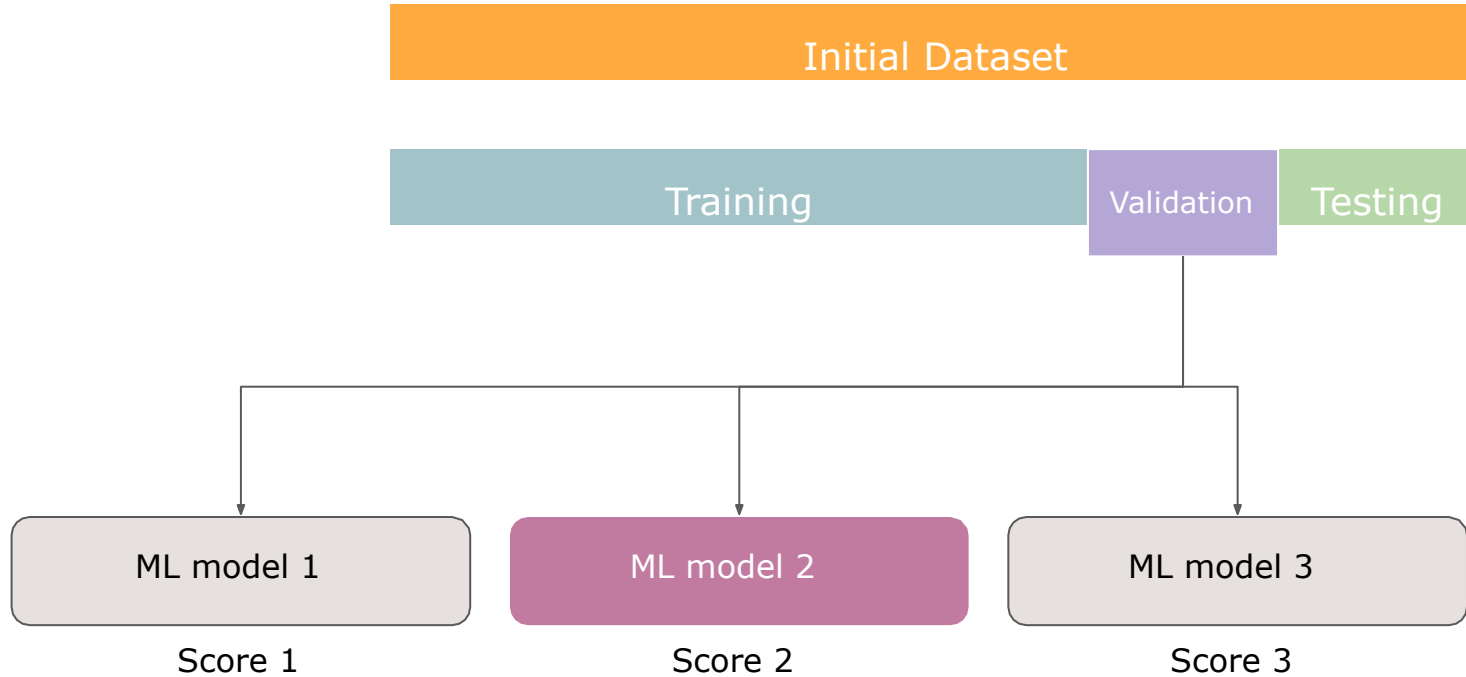


Find the best hyperparameters of your model

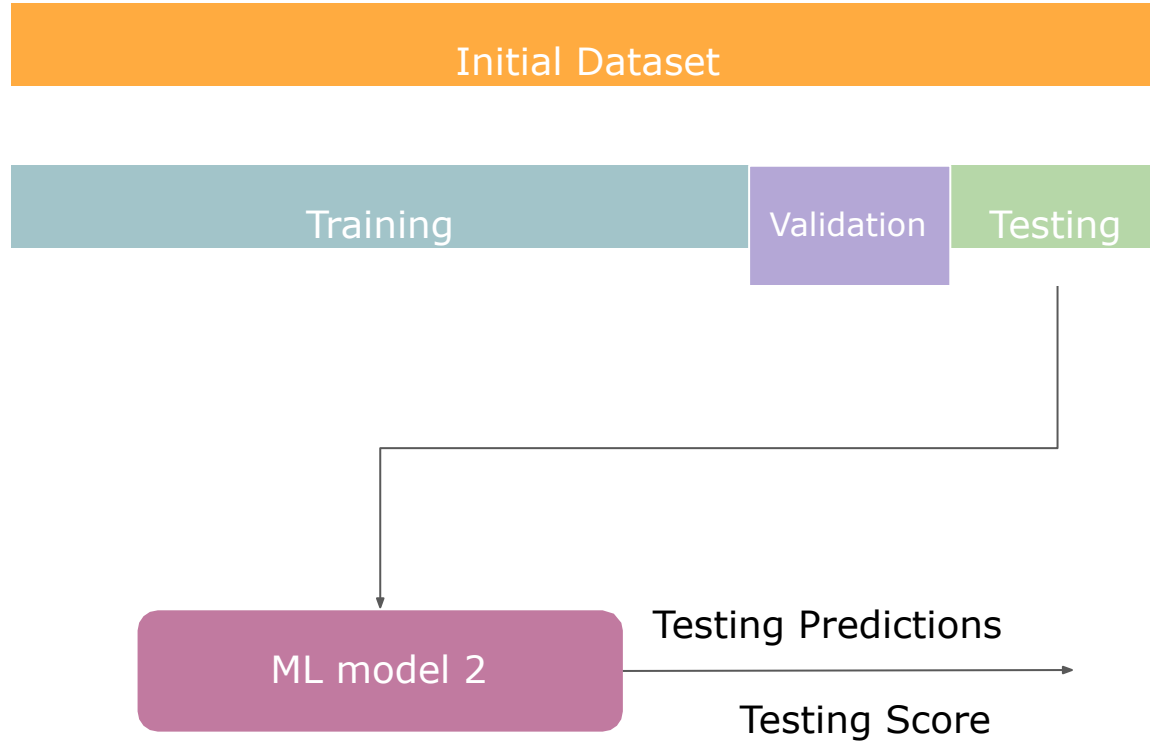
Hyperparameter Tuning



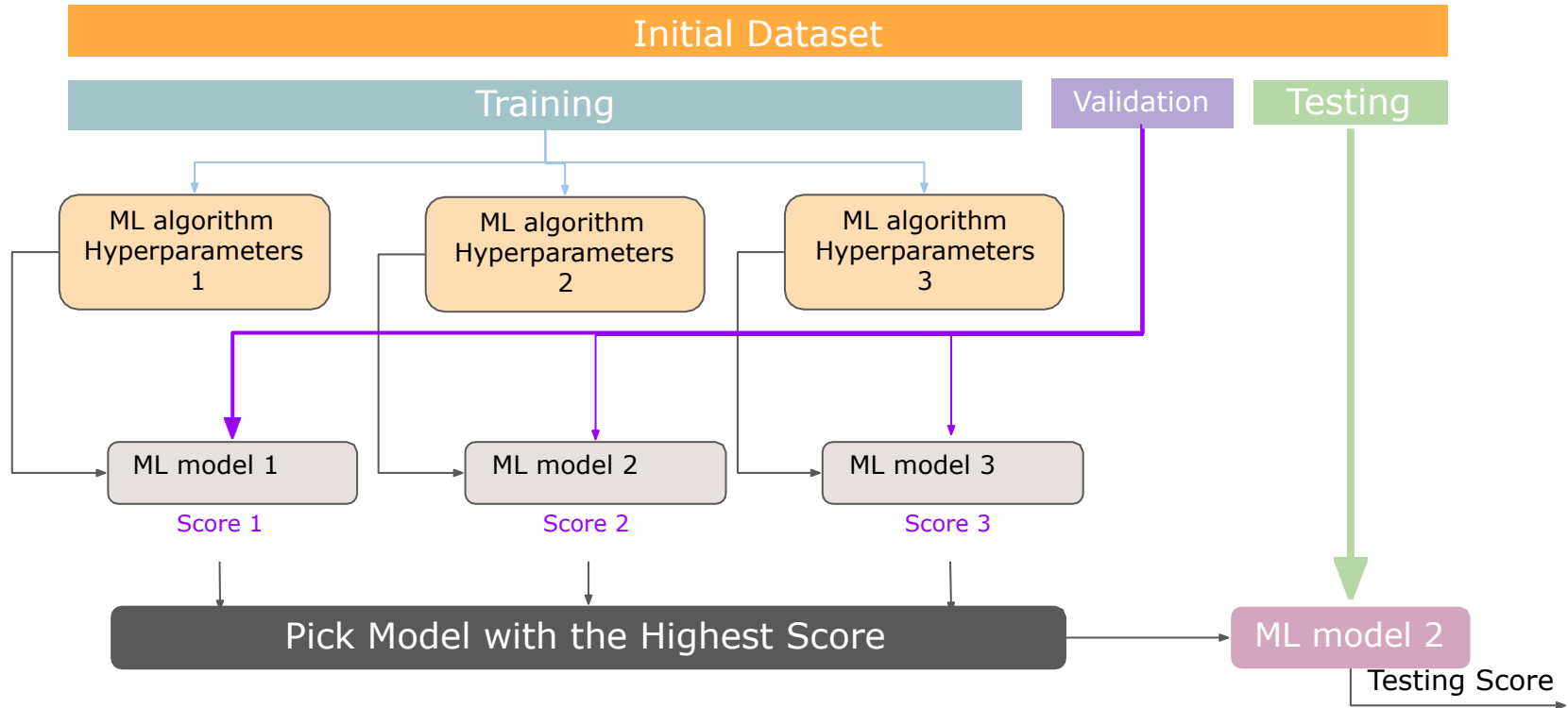
Hyperparameter Tuning



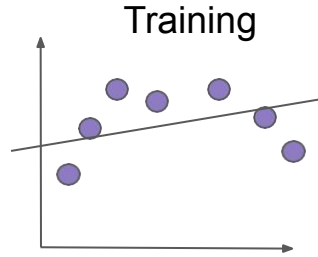
Hyperparameter Tuning



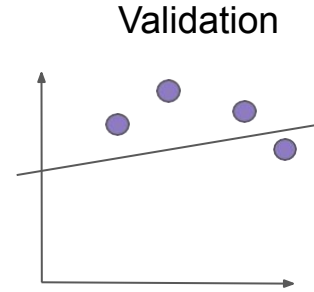
Hyperparameter Tuning



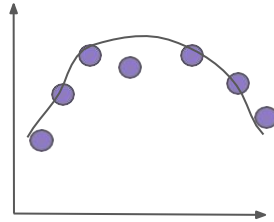
Hyperparameter Tuning



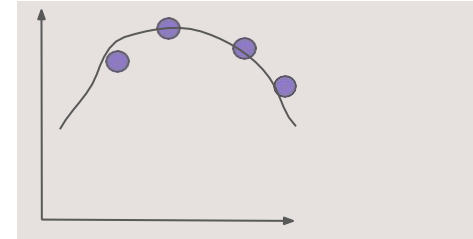
Degree = 1



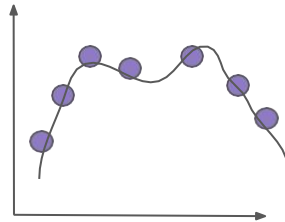
MSE1



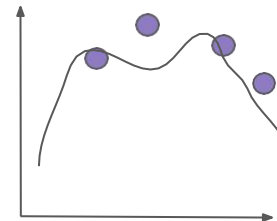
Degree = 2



We use this
model for testing
(because it has
the lowest MSE)

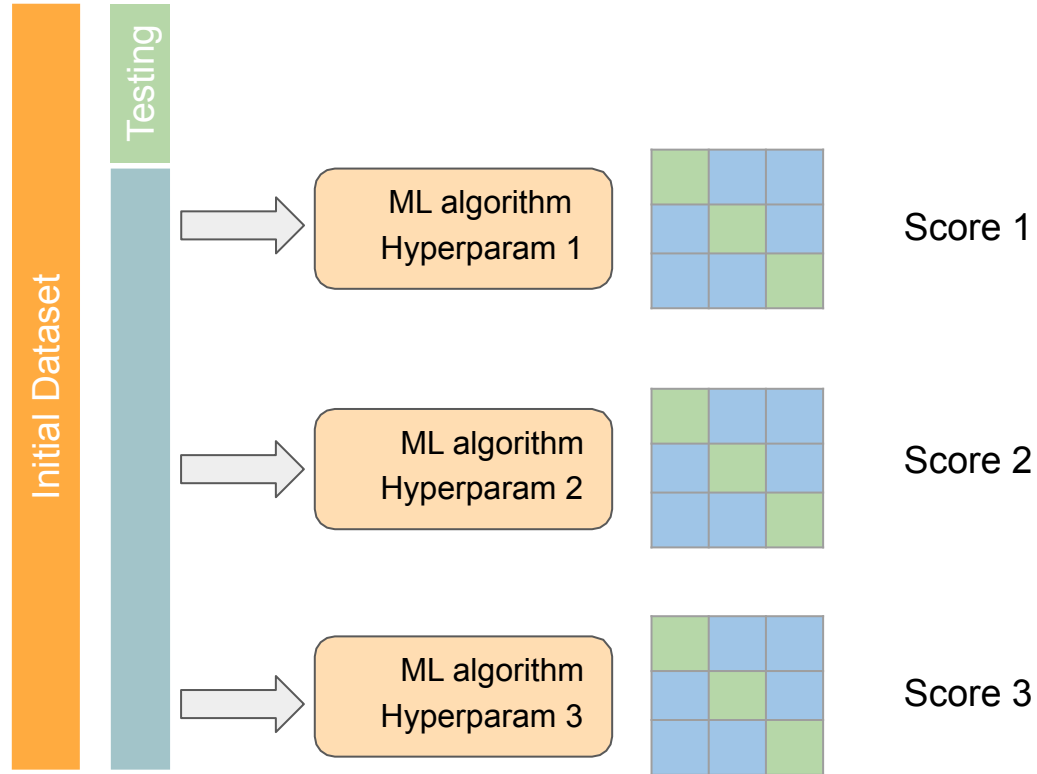


Degree = 4

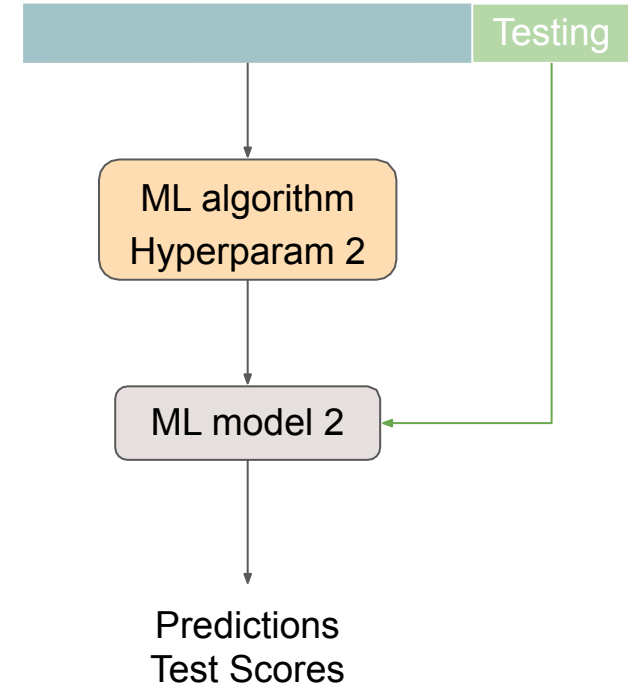
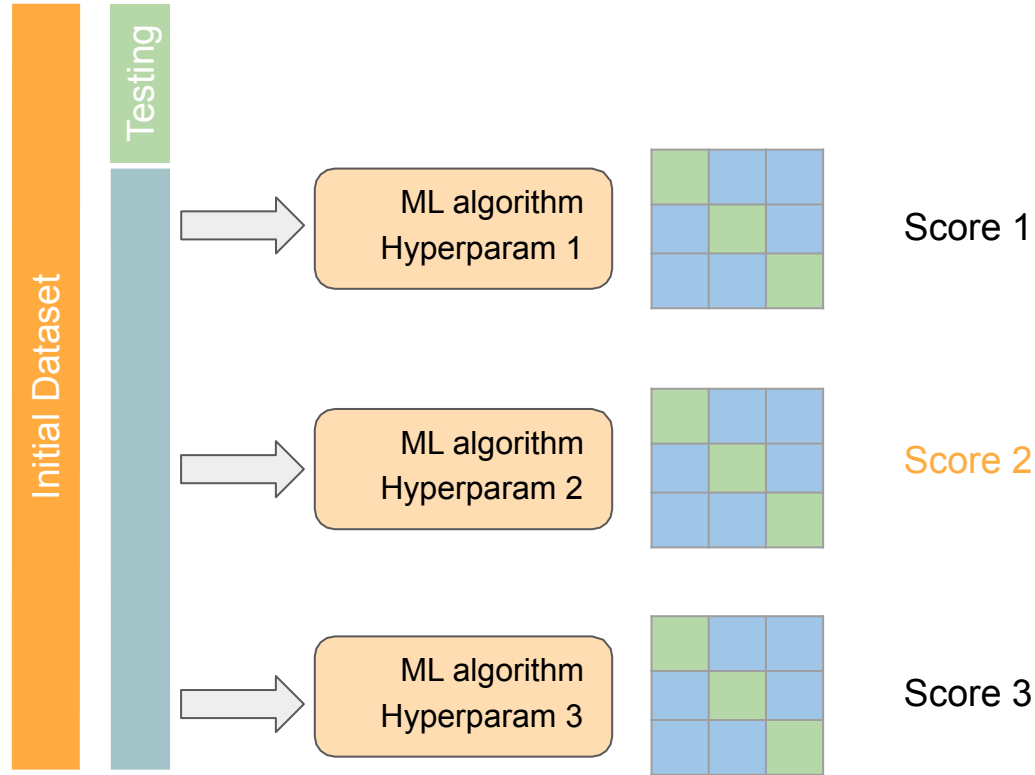


MSE3

Hyperparameter Tuning

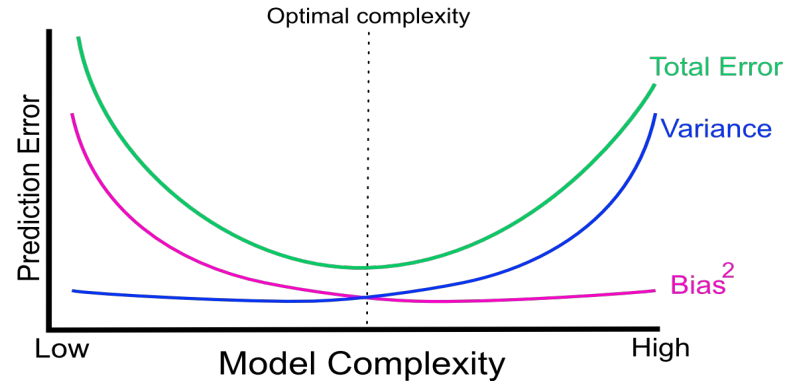
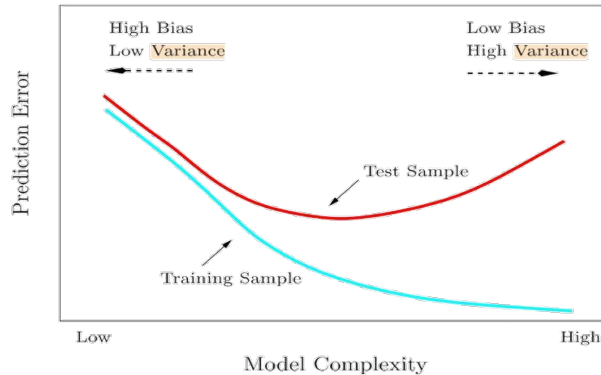


Hyperparameter Tuning



Regularization

- In gradient descent, our goal is not only to minimize the errors while building our models. We need to make models simpler (reduce their complexity).
- This is why our cost functions should be modified to take complexity into consideration.



Regularization



AMERICAN
UNIVERSITY
OF BEIRUT

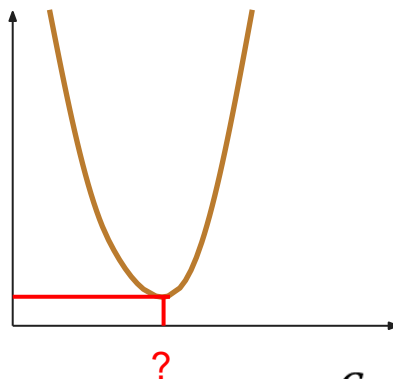
- To reach an optimal model that does not overfit, we need to add a term to the cost function that ensures our model will have optimum complexity.

Old Cost Function = Error

New Cost Function = Error + Complexity

Intuition behind Regularization

- The concept remains the same: We want to minimize the cost function.



Cost Function = Error

Minimizing the old cost function leads to **minimize the error only**

High School Again !!



Cost Function = Error + Complexity

Minimizing the new cost function leads to **minimize the error and the complexity**



How to measure complexity?

- One of the common things that reflect the complexity of a model is the number of features (consequently, *parameters*) that we have.
- As we increase those features, the chance of overfitting becomes more important.

Thus we can either:

- Reduce The Number of Features
- Keep all the features and apply regularization

Regularization

- Think of regularization as a way to keep all the features but prevent their importance from being large.
- Feature Importance is reflected by the values of their coefficients.
- The complexity term to be minimized should thus reflect these values.



Regularization

Let's say we need to minimize the effect of the feature x_2 by reducing the corresponding parameter a_2 . One good thing to do to modify the cost function is something along the following lines:

$$y_{pred} = a_0 + a_1 x_1 + a_2 x_2 + \dots + a_p x_p$$

$$Cost = MSE + 1000 a_2$$

Multiply a_2 by a large number

Minimizing the cost would lead to minimizing the value of a_2 since it's multiplied by a large number, and thus we reduce the importance of x_2 without removing it!



Regularization – Ridge (L2)

- Suppose we have a model with p parameters.
- The complexity term is reflected by the squared sum of the p parameters multiplied by a regularization factor - Lambda.
- The Lambda (Regularization factor) penalizes large parameter values.

New Cost Function = Error + Complexity

$$\text{Complexity} = \lambda \sum_{j=1}^p (a_j)^2$$



Regularization – Lasso (L1)

- Suppose we have a model with p parameters
- The complexity term is reflected by the sum of the absolute value of the p parameters multiplied by a regularization factor - Lambda.
- The Lambda (Regularization factor) factor penalizes large parameter values.

New Cost Function = Error + Complexity

$$\text{Complexity} = \lambda \sum_{j=1}^p |a_j|$$

Regularization – Linear Regression

$$y_{pred} = a_0 + a_1 x_1 + a_2 x_2 + \dots + a_p x_p$$

$$\text{Cost Function} = \text{Error} + \text{Complexity}$$

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n \left(y - (a_0 + a_1 x_1 + a_2 x_2 + \dots + a_p x_p) \right) + \frac{\lambda}{n} \sum_{i=1}^p |a_i|$$

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n \left(y - (a_0 + a_1 x_1 + a_2 x_2 + \dots + a_p x_p) \right) + \frac{\lambda}{n} \sum_{i=1}^p (a_i)^2$$

Regularization – Logistic Regression

$$y_{pred} = \sigma(a_0 + a_1 x_1 + a_2 x_2 + \dots + a_p x_p)$$

$$Cost Function = Error + Complexity$$

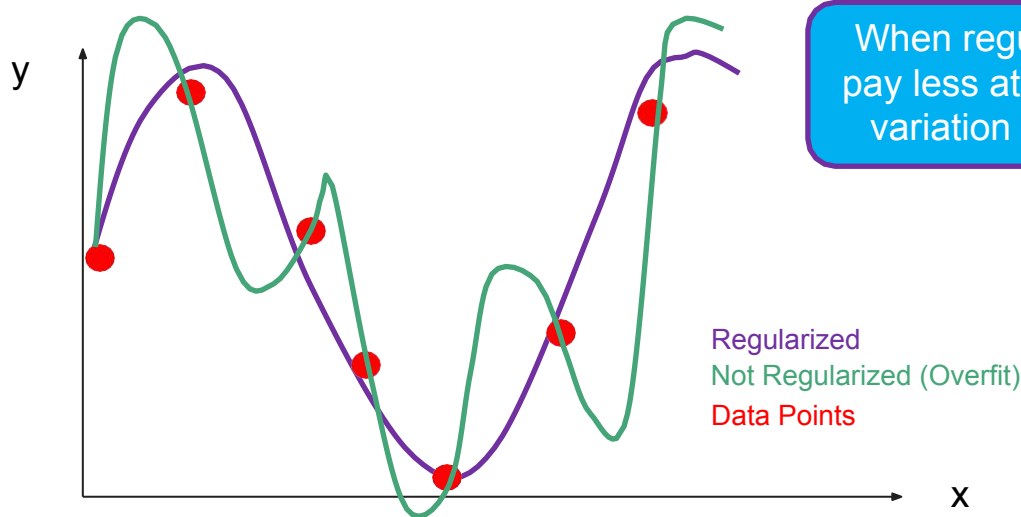
$$Cost = \frac{-1}{n} \sum_{i=1}^n y_i \log(\sigma(a_0 + a_1 x_1 + \dots + a_p x_p)) + (1 - y_i) \log(1 - \sigma(a_0 + a_1 x_1 + \dots + a_p x_p)) + \frac{\lambda}{n} \sum_{i=1}^p |a_i|$$

$$Cost = \frac{-1}{n} \sum_{i=1}^n y_i \log(\sigma(a_0 + a_1 x_1 + \dots + a_p x_p)) + (1 - y_i) \log(1 - \sigma(a_0 + a_1 x_1 + \dots + a_p x_p)) + \frac{\lambda}{n} \sum_{i=1}^p (a_i)^2$$

Effects of regularization



- Let's say we want to fit a polynomial to some data points.
- The features are $x, x^2, x^3, \dots, x^p$, where p is the order of the polynomial.



When regularizing, we pay less attention to the variation of the data.



Regularization Factor Effect

$\lambda = 0$:

- The penalty term has no effect.
- The cost function focuses on reducing the error only.

As $\lambda \rightarrow \infty$:

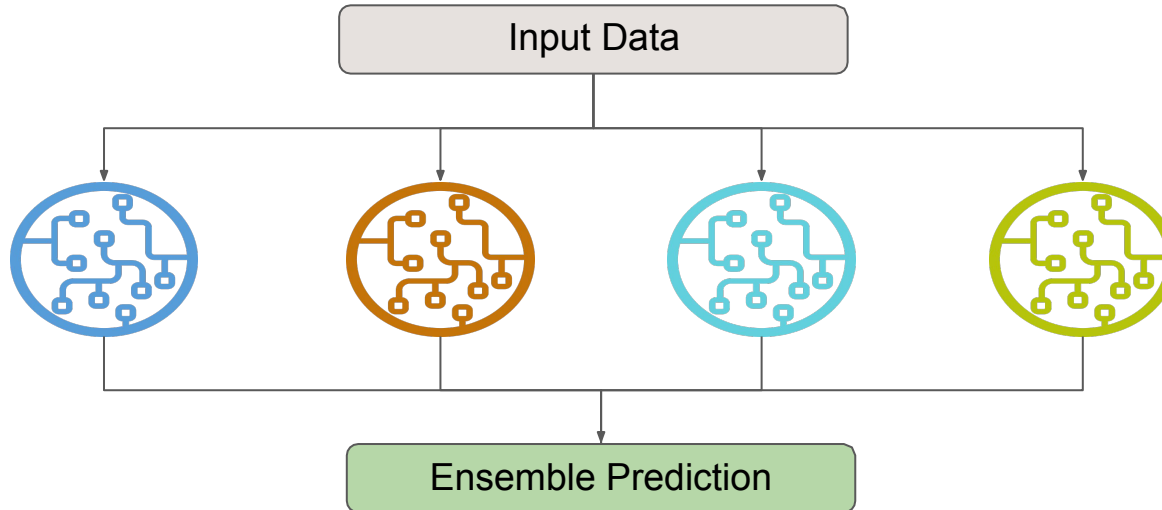
- The impact of the shrinkage penalty grows.
- The model parameters become close to zero.

Selecting a good value of λ is critical..

Improving Model Performance through Algorithm Choice

Ensemble Methods

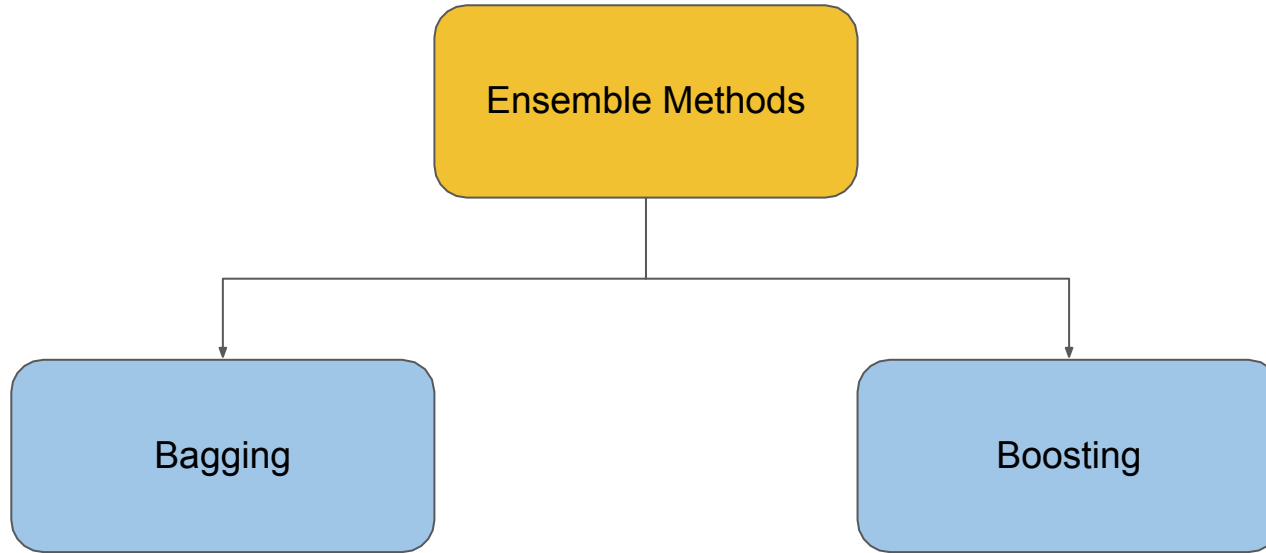
The ensemble method is a technique that relies on the aggregate prediction of several models instead of just one.



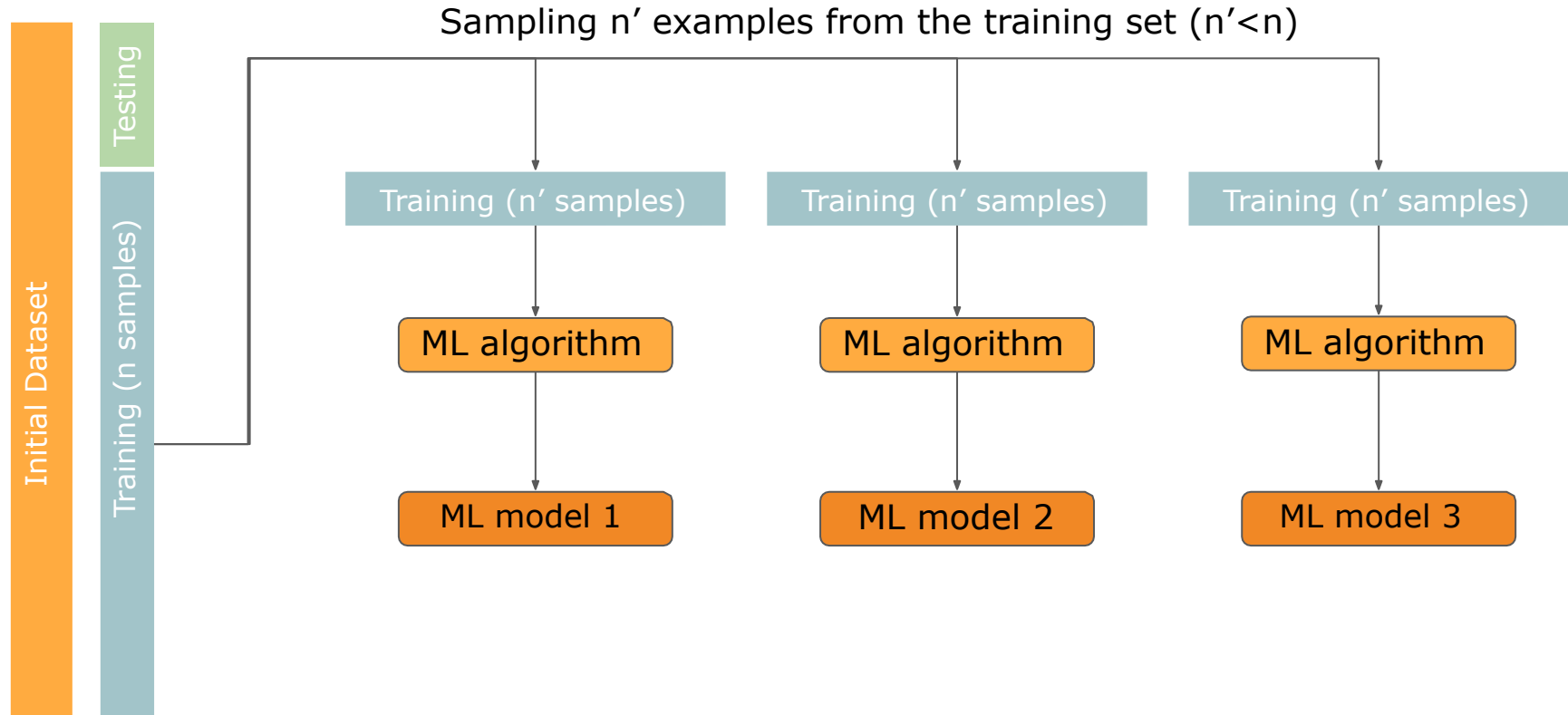
Ensemble Methods



AMERICAN
UNIVERSITY
OF BEIRUT

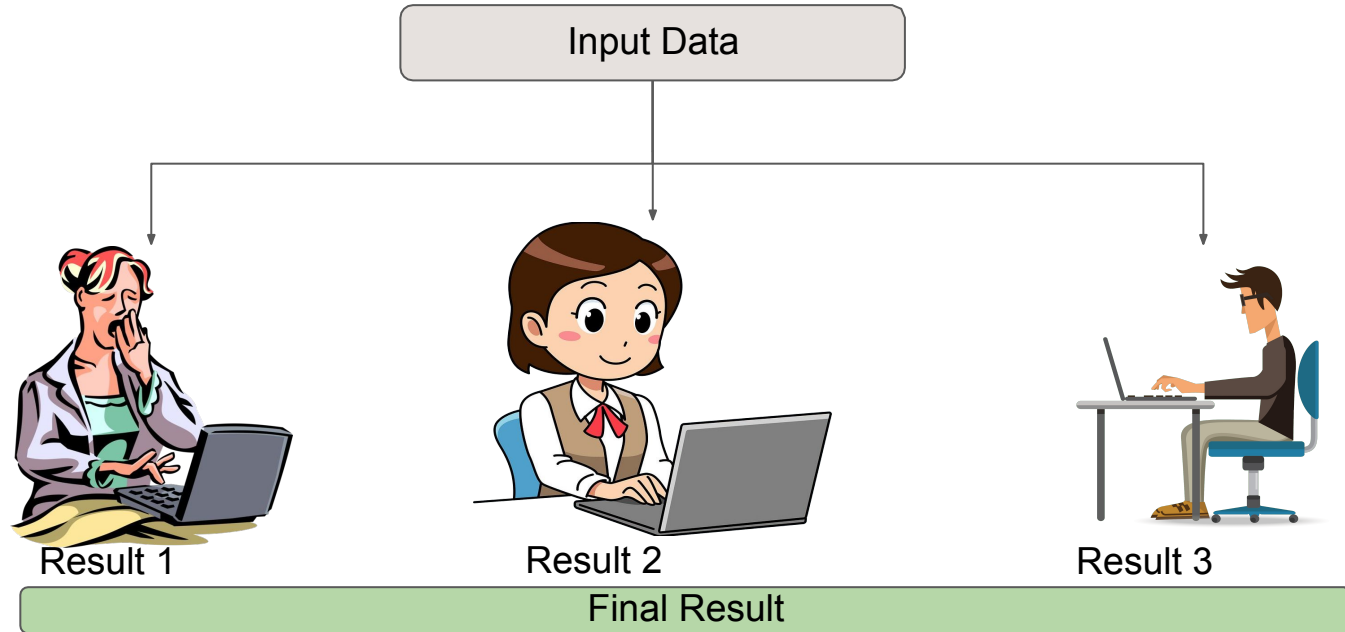


Bagging



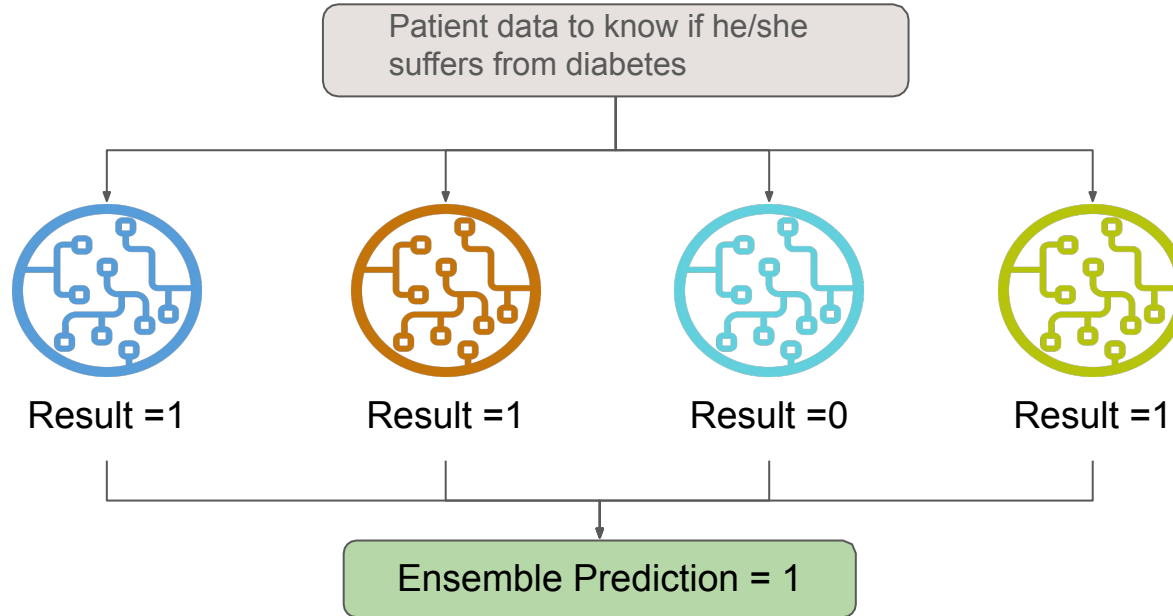
Bagging

In Bagging, models work in parallel (Separately)



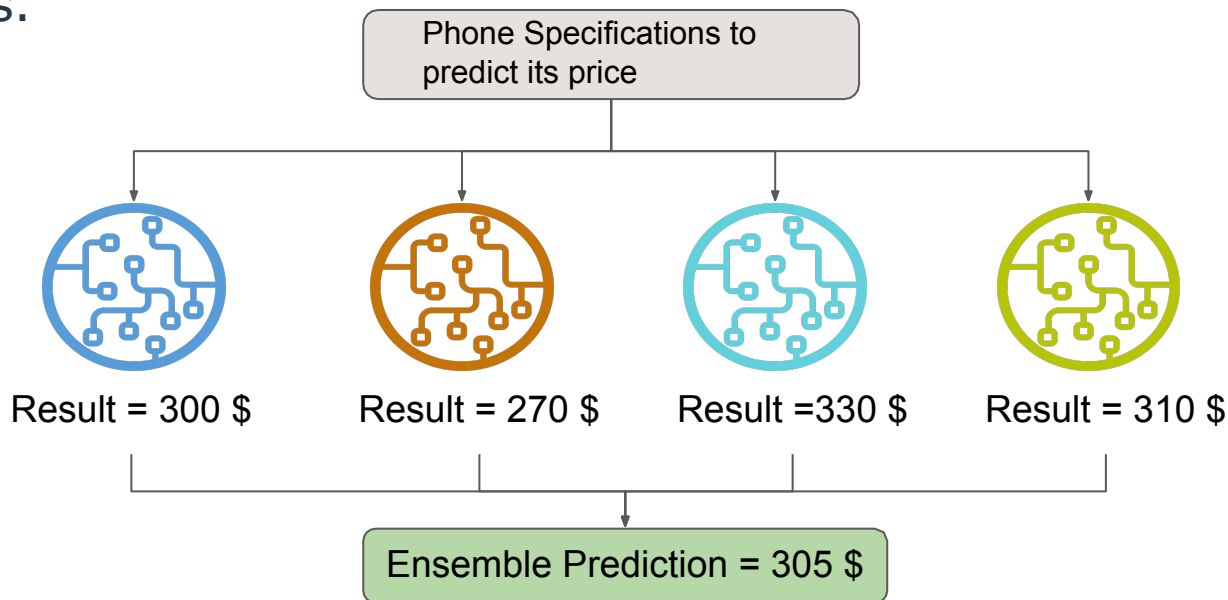
Majority Voting

- This is used in classification models.
- We choose the class that is predicted the most.



Averaging

- This is used in regression models.
- We average the values that we get from each of the models.



Boosting

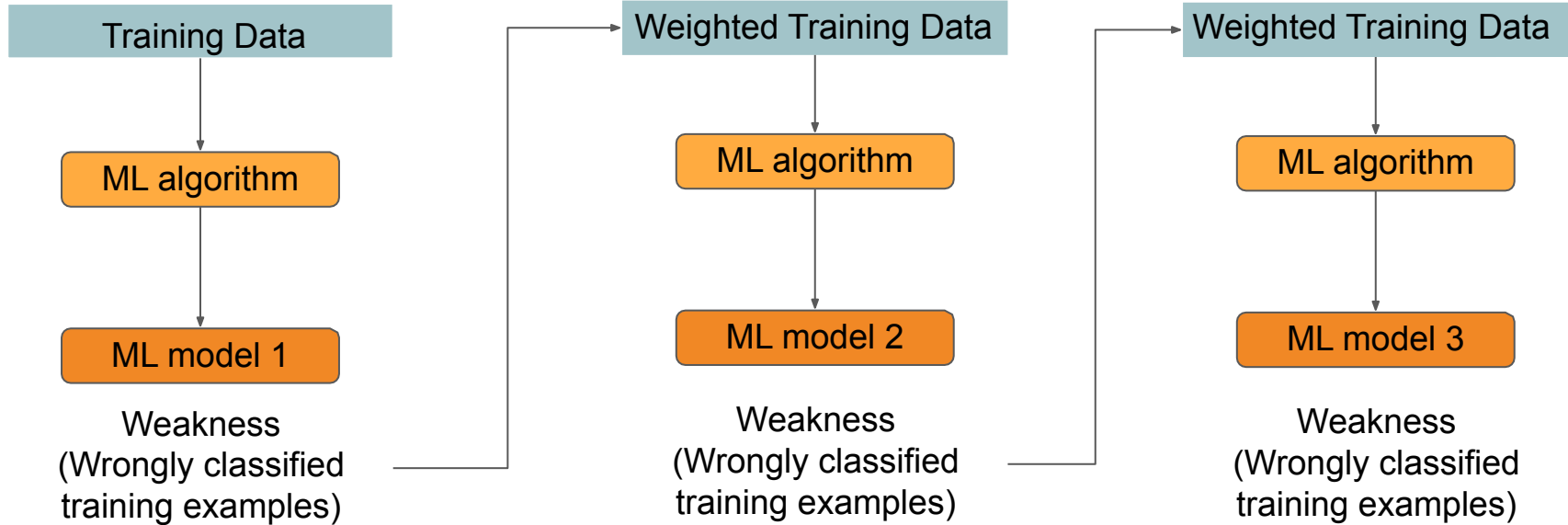


AMERICAN
UNIVERSITY
OF BEIRUT

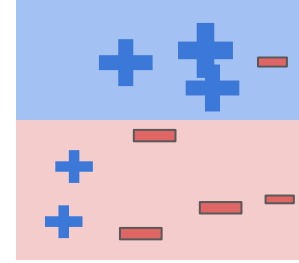
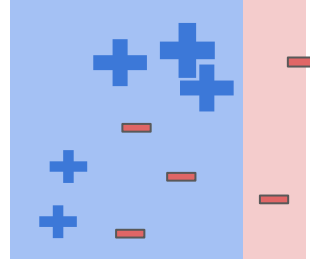
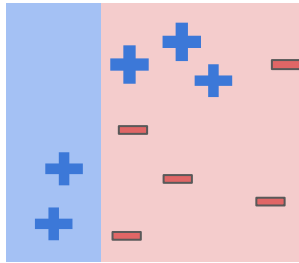
In Boosting, the individual models work sequentially - helping and correcting each others' mistakes.



Boosting

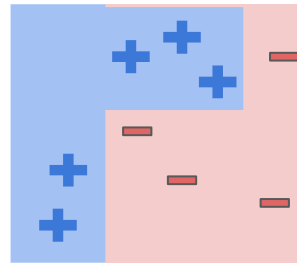


Boosting



Weak models

Aggregate results



Strong model

Improving ML Models lab in ‘Evaluation of Supervised Learning Algorithms’ Section

3



Thank You